Release Document

Status: Completed

Whenever developers push code changes or manually select a service for deployment, the CI/CD pipeline performs upstream quality gates, builds Docker images, runs image security scans, publishes approved images to DockerHub, and then proceeds with deployment operations.

This release document is written in an internal operational format and is intended to be executed step-by-step.

SECTION 1: Infrastructure Setup (Kubernetes Cluster + NFS + HAProxy + SonarQube)

This section defines the complete baseline infrastructure required to run the E-Commerce platform on Kubernetes, including cluster provisioning, shared storage, load balancing, and the upstream code-quality server used by CI quality gates.

1.1 Architecture Overview

```
┌──────────────────────────────────────────┐
│ GITHUB ACTIONS (CI/CD)                   │
│                                          │
│ ┌──────────┐  ┌──────────┐  ┌──────────┐  │
│ │ Build    │→ │ SonarQube │→ │ Docker   │  │
│ │ & Test   │  │ Scan     │  │ Build    │  │
│ └──────────┘  └──────────┘  └──────┬────┘  │
└──────────────────────────────────┬──────┘
                                   │
                                   │ kubectl apply
                                   ▼
┌──────────────────────────────────────────────────────────────┐
│ KUBERNETES CLUSTER                                            │
│                                                               │
│ ┌──────────────────────────────────────────┐                  │
│ │ MASTER NODE (t3.medium)                  │                  │
│ │                                          │                  │
│ │ • kube-apiserver   • kube-scheduler      │                  │
│ │ • kube-controller  • etcd                │                  │
│ │ • CoreDNS          • Calico CNI          │                  │
│ └──────────────────────────────────────────┘                  │
│                                                               │
│ ┌──────────────┐                  ┌──────────────┐           │
│ │ WORKER-1     │                  │ WORKER-2     │           │
│ │ (t3.medium)  │                  │ (t3.medium)  │           │
│ │              │                  │              │           │
│ │ • kubelet    │                  │ • kubelet    │           │
│ │ • kube-proxy │                  │ • kube-proxy │           │
│ │ • containerd │                  │ • containerd │           │
│ │              │                  │              │           │
│ │ Pods:        │                  │ Pods:        │           │
│ │ - frontend   │                  │ - frontend   │           │
│ │ - backend-api │                  │ - backend-api │           │
│ └──────────────┘                  └──────────────┘           │
└──────────────────────────────────────────────────────────────┘
            │                                  │
            │ NodePort:30080/30081             │ NFS Mount
            ▼                                  ▼
┌──────────────────────┐           ┌──────────────────────┐
│ NFS SERVER + HAProxy │           │ NFS Storage          │
│ (t3.medium)          │           │                      │
│                      │           │ /srv/nfs/k8s-pvs/    │
│ HAProxy:             │           │ ├── products/        │
```

```
│ Port 80  → Workers       │           │ ├── orders/              │
│ Port 443 → Workers       │           │ ├── users/               │
│ Port 8404→ Stats         │           │ ├── logs/                │
│                          │           │ └── shared/              │
│ NFS: Port 2049           │           │                         │
└──────────────────────────┘           └─────────────────────────┘
             │
             │
             ▼
        ┌──────────┐
        │ USERS    │
        │(Internet) │
        └──────────┘


┌──────────────────────────┐
│ SONARQUBE SERVER         │
│ (t3.medium)              │
│                          │
│ Port: 9000               │
│ H2 Embedded Database      │
│ Code Quality Analysis     │
└──────────────────────────┘
```

1.2 Infrastructure Inventory

#	Server	Role	Instance Type	Private IP	OS	Key Ports
---:	---	---	---	---	---	---
1	master-node	K8s Control Plane	t3.medium	172.31.1.10	Ubuntu 22.04 LTS	6443, 2379-2380, 10250-10252
2	worker-node-1	K8s Worker	t3.medium	172.31.1.11	Ubuntu 22.04 LTS	10250, 30000-32767
3	worker-node-2	K8s Worker	t3.medium	172.31.1.12	Ubuntu 22.04 LTS	10250, 30000-32767
4	nfs-haproxy	NFS + Load Balancer	t3.medium	172.31.1.13	Ubuntu 22.04 LTS	2049, 80, 443, 8404
5	sonarqube	Code Quality	t3.medium	172.31.1.14	Ubuntu 22.04 LTS	9000

Note: Replace all example IPs with your actual AWS EC2 private IPs throughout this document.

1.3 Pre-requisites & AWS Security Groups

Configure the following Security Groups in AWS Console before starting:

SG: sg-k8s-master

Inbound Rules:

- TCP `6443` → `172.31.0.0/16` (K8s API Server)
- TCP `2379-2380` → `172.31.0.0/16` (etcd)
- TCP `10250-10252` → `172.31.0.0/16` (Kubelet/Controller)
- TCP `22` → Your IP (SSH)

SG: sg-k8s-worker

Inbound Rules:

- TCP `10250` → `172.31.0.0/16` (Kubelet)
- TCP `30000-32767` → `172.31.0.0/16` (NodePort Services)
- TCP `22` → Your IP (SSH)

SG: sg-nfs-haproxy

Inbound Rules:

- TCP `2049` → `172.31.0.0/16` (NFS)
- TCP `111` → `172.31.0.0/16` (RPC portmapper)
- TCP `80` → `0.0.0.0/0` (HTTP via HAProxy)
- TCP `443` → `0.0.0.0/0` (HTTPS via HAProxy)
- TCP `8404` → Your IP (HAProxy Stats)
- TCP `22` → Your IP (SSH)

SG: sg-sonarqube

Inbound Rules:

- TCP `9000` → `0.0.0.0/0` (SonarQube UI)
- TCP `22` → Your IP (SSH)

1.4 Phase 1: Kubernetes Cluster Setup

1.4.1 Setup Order

- Step 1 → All 3 Nodes (Common Setup)
- Step 2 → Master Node Only (`kubeadm init`)
- Step 3 → Worker Nodes (`kubeadm join`)

1.4.2 Common Setup — Run on ALL 3 Nodes (Master + Worker1 + Worker2)

SSH into each node and run the following commands one by one.

Step 1: Update System Packages

```
sudo apt update && sudo apt upgrade -y
```

Step 2: Disable Swap

Kubernetes requires swap to be disabled. If swap is enabled, kubelet will not start.

```
# Disable swap immediately (for current session)
sudo swapoff -a

# Disable swap permanently (survives reboot)
# This comments out the swap line in /etc/fstab
sudo sed -i '/ swap / s/^/#/' /etc/fstab
```

```
# Verify swap is completely off
free -h
```

Expected Output:

```
              total        used        free
Swap:            0B          0B          0B
```

Step 3: Load Required Kernel Modules

These modules are required for Kubernetes networking to work properly.

```
# Create the modules config file
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

# Load the modules immediately (without reboot)
sudo modprobe overlay
sudo modprobe br_netfilter

# Verify modules are loaded
lsmod | grep br_netfilter
lsmod | grep overlay
```

Expected Output:

```
br_netfilter 28672 0
bridge 307200 1 br_netfilter
overlay 151552 0
```

Step 4: Configure Kernel Network Parameters

These settings enable iptables to see bridged traffic and enable IP forwarding.

```
# Write the sysctl configuration
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF

# Apply settings immediately without reboot
sudo sysctl --system

# Verify the settings are applied
```

```
sysctl net.bridge.bridge-nf-call-iptables
sysctl net.ipv4.ip_forward
```

Expected Output:

```
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
```

Step 5: Install Containerd (Container Runtime)

Kubernetes requires a container runtime. We use `containerd`, which is the industry standard.

```
# Install required packages
sudo apt install -y ca-certificates curl gnupg lsb-release apt-transport-https

# Create the directory for GPG keys
sudo install -m 0755 -d /etc/apt/keyrings

# Add Docker's official GPG key (containerd is distributed via Docker repo)
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Add Docker repository to apt sources
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Update and install containerd
sudo apt update
sudo apt install -y containerd.io

# Verify containerd installation
containerd --version
```

Step 6: Configure Containerd

This is critical. Without `SystemdCgroup` enabled, pods may crash or behave unpredictably.

```
# Generate the default configuration file
sudo mkdir -p /etc/containerd
sudo containerd config default | sudo tee /etc/containerd/config.toml > /dev/null

# Enable SystemdCgroup - REQUIRED for Kubernetes
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g' /etc/containerd/config.
toml
```

```
# Verify the change was applied correctly
grep -i "SystemdCgroup" /etc/containerd/config.toml

# Restart containerd to apply new configuration
sudo systemctl restart containerd
sudo systemctl enable containerd

# Verify containerd is running
sudo systemctl status containerd
```

Expected Output:

```
SystemdCgroup = true
```

Step 7: Install Kubernetes Components

We install:

- `kubeadm` — bootstraps the cluster
- `kubelet` — runs on every node, manages pods
- `kubectl` — CLI tool to interact with cluster

```
# Add Kubernetes official GPG key
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

# Add Kubernetes repository
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | \
  sudo tee /etc/apt/sources.list.d/kubernetes.list

# Update package list
sudo apt update

# Install kubelet, kubeadm, kubectl
sudo apt install -y kubelet kubeadm kubectl

# Pin the versions to prevent accidental upgrades (stability requirement)
sudo apt-mark hold kubelet kubeadm kubectl

# Verify all three are installed
kubeadm version
kubectl version --client
kubelet --version
```

Step 8: Install NFS Client

Required on all worker nodes so they can mount NFS volumes from the NFS server.

```
sudo apt install -y nfs-common
```

Step 9: Set Hostname and Update /etc/hosts

On Master Node only:

```
sudo hostnamectl set-hostname master-node
```

On Worker Node 1 only:

```
sudo hostnamectl set-hostname worker-node-1
```

On Worker Node 2 only:

```
sudo hostnamectl set-hostname worker-node-2
```

On ALL 3 nodes — add entries to `/etc/hosts`:

```
sudo bash -c 'cat >> /etc/hosts << EOF
# Kubernetes Cluster Nodes
172.31.1.10 master-node
172.31.1.11 worker-node-1
172.31.1.12 worker-node-2
172.31.1.13 nfs-server
EOF'
cat /etc/hosts
```

1.4.3 Master Node Setup — Run ONLY on Master Node

SSH into the Master Node:

```
ssh -i "your-key.pem" ubuntu@<master-node-public-ip>
```

Step 1: Initialize the Kubernetes Cluster

```
sudo kubeadm init \
  --pod-network-cidr=10.244.0.0/16 \
  --apiserver-advertise-address=172.31.1.10 \
  --kubernetes-version=v1.29.0 \
  --control-plane-endpoint=172.31.1.10:6443
```

What each flag means:

Flag	Purpose
`--pod-network-cidr`	IP range for pod network (must match Calico config)

Flag	Purpose
`--apiserver-advertise-address`	Master node's private IP
`--kubernetes-version`	Pin to a specific version
`--control-plane-endpoint`	Endpoint for the API server

Important: Copy and save the `kubeadm join` command from the output. You will need it for the worker nodes.

Step 2: Configure kubectl Access

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

kubectl get nodes
```

Expected Output (before CNI):

```
NAME         STATUS     ROLES           AGE   VERSION
master-node  NotReady   control-plane   1m    v1.29.0
```

Step 3: Install Calico CNI (Pod Network Plugin)

Without a CNI plugin, pods cannot communicate with each other.

```
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.27.0/mani
fests/calico.yaml

kubectl get pods -n kube-system -w
```

Wait until pods show `Running`, then verify nodes:

```
kubectl get nodes
```

Expected Output:

```
NAME         STATUS   ROLES           AGE   VERSION
master-node  Ready    control-plane   5m    v1.29.0
```

Step 4: Generate Join Token (In Case You Need It Later)

Tokens expire in 24 hours. If the join command expires, generate a new one:

```
kubeadm token create --print-join-command
```

1.4.4 Worker Nodes Setup — Run on Worker-1 and Worker-2

SSH into each worker node:

```
# Terminal 1 - Worker Node 1
ssh -i "your-key.pem" ubuntu@<worker-1-public-ip>

# Terminal 2 - Worker Node 2
ssh -i "your-key.pem" ubuntu@<worker-2-public-ip>
```

Step 1: Join the Cluster

Use the join command from `kubeadm init` output:

```
sudo kubeadm join 172.31.1.10:6443 --token abcdef.1234567890abcdef \
  --discovery-token-ca-cert-hash sha256:abc123def456...
```

1.4.5 Verify Cluster — Run on Master Node

```
kubectl get nodes -o wide
```

Expected Output:

```
NAME           STATUS   ROLES           AGE   VERSION   INTERNAL-IP   OS-IMAGE
master-node    Ready    control-plane   10m   v1.29.0   172.31.1.10    Ubuntu 22.04
worker-node-1  Ready    <none>          3m    v1.29.0   172.31.1.11    Ubuntu 22.04
worker-node-2  Ready    <none>          2m    v1.29.0   172.31.1.12    Ubuntu 22.04
```

Label worker nodes for identification:

```
kubectl label node worker-node-1 node-role.kubernetes.io/worker=worker
kubectl label node worker-node-2 node-role.kubernetes.io/worker=worker
kubectl get nodes
```

Verify system pods:

```
kubectl get pods -n kube-system
kubectl cluster-info
```

1.5 Phase 2: NFS Server + HAProxy Setup

Both NFS and HAProxy are installed on the same EC2 instance (`t3.medium`).

1.5.1 SSH into NFS + HAProxy Server

```
ssh -i "your-key.pem" ubuntu@<nfs-haproxy-public-ip>
```

1.5.2 Part A: NFS Server Setup

Step 1: Update System

```
sudo apt update && sudo apt upgrade -y
```

Step 2: Install NFS Server Package

```
sudo apt install -y nfs-kernel-server nfs-common
sudo systemctl status nfs-kernel-server
```

Step 3: Create NFS Export Directories

We create separate directories for each e-commerce workload category:

```
# Create base directory
sudo mkdir -p /srv/nfs/k8s-pvs

# Create service-specific directories
sudo mkdir -p /srv/nfs/k8s-pvs/ecommerce-products     # Product images & catalog dat
a
sudo mkdir -p /srv/nfs/k8s-pvs/ecommerce-orders       # Order records
sudo mkdir -p /srv/nfs/k8s-pvs/ecommerce-users        # User uploads & avatars
sudo mkdir -p /srv/nfs/k8s-pvs/ecommerce-logs         # Application logs
sudo mkdir -p /srv/nfs/k8s-pvs/ecommerce-db-backup    # Database backups
sudo mkdir -p /srv/nfs/k8s-pvs/ecommerce-shared       # Shared assets & config

# Set permissions
sudo chown -R nobody:nogroup /srv/nfs/k8s-pvs
sudo chmod -R 777 /srv/nfs/k8s-pvs

# Verify directory structure
ls -la /srv/nfs/k8s-pvs/
```

Expected Output (structure):

- `ecommerce-db-backup`
- `ecommerce-logs`
- `ecommerce-orders`
- `ecommerce-products`
- `ecommerce-shared`
- `ecommerce-users`

Step 4: Configure NFS Exports

```
sudo nano /etc/exports
```

Add the following configuration:

```
# /etc/exports - NFS Export Configuration
# Format: directory client(options)
# Replace 172.31.0.0/16 with your actual VPC CIDR
/srv/nfs/k8s-pvs/ecommerce-products    172.31.0.0/16(rw,sync,no_subtree_check,no_roo
t_squash)
/srv/nfs/k8s-pvs/ecommerce-orders      172.31.0.0/16(rw,sync,no_subtree_check,no_roo
t_squash)
/srv/nfs/k8s-pvs/ecommerce-users       172.31.0.0/16(rw,sync,no_subtree_check,no_roo
t_squash)
/srv/nfs/k8s-pvs/ecommerce-logs        172.31.0.0/16(rw,sync,no_subtree_check,no_roo
t_squash)
/srv/nfs/k8s-pvs/ecommerce-db-backup   172.31.0.0/16(rw,sync,no_subtree_check,no_roo
t_squash)
/srv/nfs/k8s-pvs/ecommerce-shared      172.31.0.0/16(rw,sync,no_subtree_check,no_roo
t_squash)
```

NFS Export Options Explained:

Option	Description
`rw`	Allow read and write operations
`sync`	Write data to disk before replying (data safe)
`no_subtree_check`	Disables subtree checking, improves reliability
`no_root_squash`	Allows root on clients to write as root on server

Step 5: Apply NFS Export Configuration

```
sudo exportfs -arv
```

Expected Output (example):

- `exporting 172.31.0.0/16:/srv/nfs/k8s-pvs/ecommerce-products`
- `exporting 172.31.0.0/16:/srv/nfs/k8s-pvs/ecommerce-orders`
- `exporting 172.31.0.0/16:/srv/nfs/k8s-pvs/ecommerce-users`
- `exporting 172.31.0.0/16:/srv/nfs/k8s-pvs/ecommerce-logs`
- `exporting 172.31.0.0/16:/srv/nfs/k8s-pvs/ecommerce-db-backup`
- `exporting 172.31.0.0/16:/srv/nfs/k8s-pvs/ecommerce-shared`

Step 6: Start and Enable NFS Server

```
sudo systemctl enable nfs-kernel-server
sudo systemctl restart nfs-kernel-server
sudo systemctl status nfs-kernel-server
```

Step 7: Verify NFS Server

```
sudo exportfs -v
sudo ss -tlnp | grep 2049
showmount -e localhost
```

Expected Output of `showmount -e localhost`:

- `/srv/nfs/k8s-pvs/ecommerce-shared 172.31.0.0/16`
- `/srv/nfs/k8s-pvs/ecommerce-db-backup 172.31.0.0/16`
- `/srv/nfs/k8s-pvs/ecommerce-logs 172.31.0.0/16`
- `/srv/nfs/k8s-pvs/ecommerce-users 172.31.0.0/16`
- `/srv/nfs/k8s-pvs/ecommerce-orders 172.31.0.0/16`
- `/srv/nfs/k8s-pvs/ecommerce-products 172.31.0.0/16`

Step 8: Test NFS from Kubernetes Worker Nodes

On each worker node:

```
# Check what NFS server is exporting
showmount -e 172.31.1.13

# Test mounting an NFS share
sudo mount -t nfs 172.31.1.13:/srv/nfs/k8s-pvs/ecommerce-products /mnt
ls /mnt
sudo umount /mnt
```

If the mount succeeds, NFS is working correctly.

1.5.3 Part B: HAProxy Setup

HAProxy acts as a load balancer, distributing incoming user traffic across both Kubernetes worker nodes.

Step 1: Install HAProxy

```
sudo apt install -y haproxy
haproxy -v
```

Step 2: Backup Default Configuration

```
sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.backup
```

Step 3: Configure HAProxy

```
sudo nano /etc/haproxy/haproxy.cfg
```

Replace the entire contents with:

```
#---------------------------------------------------------------------
# Global Settings
#---------------------------------------------------------------------
global
  log /dev/log local0
  log /dev/log local1 notice
  chroot /var/lib/haproxy
  stats socket /run/haproxy/admin.sock mode 660 level admin
  stats timeout 30s
  user haproxy
  group haproxy
  daemon

  # SSL configuration
  ca-base /etc/ssl/certs
  crt-base /etc/ssl/private
  ssl-default-bind-ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256
  ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets

#---------------------------------------------------------------------
# Default Settings
#---------------------------------------------------------------------
defaults
  log global
  mode http
  option httplog
  option dontlognull
  option forwardfor
  option http-server-close
  timeout connect 5000ms
  timeout client 50000ms
  timeout server 50000ms
  errorfile 400 /etc/haproxy/errors/400.http
  errorfile 403 /etc/haproxy/errors/403.http
  errorfile 408 /etc/haproxy/errors/408.http
  errorfile 500 /etc/haproxy/errors/500.http
  errorfile 502 /etc/haproxy/errors/502.http
  errorfile 503 /etc/haproxy/errors/503.http
  errorfile 504 /etc/haproxy/errors/504.http

#---------------------------------------------------------------------
# HAProxy Statistics Dashboard
# Access: http://<haproxy-ip>:8404/stats
#---------------------------------------------------------------------
frontend stats
  bind *:8404
  stats enable
  stats uri /stats
  stats refresh 10s
  stats auth admin:StrongPassword@123
  stats admin if LOCALHOST

#---------------------------------------------------------------------
# E-Commerce HTTP Frontend (Port 80)
#---------------------------------------------------------------------
frontend ecommerce_http_frontend
  bind *:80
  mode http

  # Security headers
  http-response set-header X-Frame-Options SAMEORIGIN
  http-response set-header X-Content-Type-Options nosniff
  http-response set-header X-XSS-Protection "1; mode=block"

  # Path-based routing using ACLs
  acl is_api path_beg /api
  acl is_admin path_beg /admin

  # Route API traffic to API backend
```

```
  use_backend ecommerce_api_backend if is_api

  # Route all other traffic to frontend
  default_backend ecommerce_frontend_backend

#---------------------------------------------------------------------
# Frontend Backend Pool
# Worker nodes expose frontend on NodePort 30080
#---------------------------------------------------------------------
backend ecommerce_frontend_backend
  mode http
  balance roundrobin
  option httpchk GET /health
  http-check expect status 200

  # Sticky sessions to keep shopping cart consistent
  cookie SERVERID insert indirect nocache

  server worker-1 172.31.1.11:30080 check cookie w1 inter 5s fall 3 rise 2
  server worker-2 172.31.1.12:30080 check cookie w2 inter 5s fall 3 rise 2

#---------------------------------------------------------------------
# API Backend Pool
# Worker nodes expose API on NodePort 30081
#---------------------------------------------------------------------
backend ecommerce_api_backend
  mode http
  balance roundrobin
  option httpchk GET /api/health
  http-check expect status 200

  server worker-1 172.31.1.11:30081 check inter 5s fall 3 rise 2
  server worker-2 172.31.1.12:30081 check inter 5s fall 3 rise 2
```

Step 4: Validate HAProxy Configuration

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
```

Expected Output:

```
Configuration file is valid
```

Step 5: Start and Enable HAProxy

```
sudo systemctl enable haproxy
sudo systemctl restart haproxy
sudo systemctl status haproxy
```

Step 6: Verify HAProxy is Listening

```
sudo ss -tlnp | grep haproxy
```

Expected Output (example):

- `*:80`
- `*:8404`

Step 7: Access HAProxy Stats Dashboard

- URL: `http://<nfs-haproxy-public-ip>:8404/stats`
- Username: `admin`
- Password: `StrongPassword@123`

1.6 Phase 3: SonarQube Server Setup

SonarQube is the code quality and security analysis server that integrates with GitHub Actions to enforce upstream quality gates before deployments.

1.6.1 SSH into SonarQube Server

```
ssh -i "your-key.pem" ubuntu@<sonarqube-public-ip>
```

Step 1: Update System

```
sudo apt update && sudo apt upgrade -y
```

Step 2: Install Java 17

```
sudo apt install -y openjdk-17-jdk unzip wget net-tools
java -version
```

Step 3: Configure System Requirements for SonarQube

SonarQube uses Elasticsearch internally and requires specific kernel settings.

```
sudo bash -c 'cat >> /etc/sysctl.conf << EOF
# SonarQube / Elasticsearch Requirements
vm.max_map_count=524288
fs.file-max=131072
EOF'

sudo sysctl -p

sysctl vm.max_map_count
sysctl fs.file-max
```

Expected Output:

- `vm.max_map_count = 524288`
- `fs.file-max = 131072`

Set system limits for the `sonarqube` user:

```
sudo bash -c 'cat >> /etc/security/limits.conf << EOF
# SonarQube User Limits
sonarqube - nofile 131072
```

```
sonarqube - nproc 8192
EOF'
```

Step 4: Create SonarQube Dedicated User

SonarQube must not run as root.

```
sudo useradd -m -d /opt/sonarqube -s /bin/bash sonarqube
id sonarqube
```

Step 5: Download and Install SonarQube

```
cd /opt

sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-10.6.0.9
2116.zip
sudo unzip sonarqube-10.6.0.92116.zip
sudo mv sonarqube-10.6.0.92116 sonarqube
sudo chown -R sonarqube:sonarqube /opt/sonarqube

ls -la /opt/sonarqube/
```

Expected Directory Structure:

- `/opt/sonarqube/bin/`
- `/opt/sonarqube/conf/`
- `/opt/sonarqube/data/` (H2 database stored here)
- `/opt/sonarqube/logs/`
- `/opt/sonarqube/extensions/`
- `/opt/sonarqube/lib/`

Step 6: Configure SonarQube

```
sudo -u sonarqube nano /opt/sonarqube/conf/sonar.properties
```

Update/add the following properties:

```
# ==============================================================
# DATABASE CONFIGURATION
# ==============================================================
# We are using H2 - the built-in embedded database
# H2 requires NO external database setup
# Data is stored in: /opt/sonarqube/data/
# DO NOT set sonar.jdbc.url - leaving it blank = H2 is used

# ==============================================================
# WEB SERVER CONFIGURATION
```

```
# ==============================================================
sonar.web.host=0.0.0.0
sonar.web.port=9000
sonar.web.context=/

# ==============================================================
# JVM MEMORY SETTINGS (Optimized for t3.medium with 4GB RAM)
# ==============================================================
sonar.web.javaOpts=-Xmx512m -Xms256m -XX:+HeapDumpOnOutOfMemoryError
sonar.ce.javaOpts=-Xmx512m -Xms256m -XX:+HeapDumpOnOutOfMemoryError
sonar.search.javaOpts=-Xmx512m -Xms512m -XX:MaxDirectMemorySize=256m

# ==============================================================
# LOGGING
# ==============================================================
sonar.log.level=INFO
sonar.path.logs=logs
```

Note on H2 Database:

- H2 is suitable for small teams and development/testing.
- H2 data is stored in `/opt/sonarqube/data/`.
- For larger teams, migrate to PostgreSQL.

Step 7: Create Systemd Service File

```
sudo nano /etc/systemd/system/sonarqube.service
```

Add:

```
[Unit]
Description=SonarQube Code Quality Server
Documentation=https://docs.sonarqube.org
After=network.target
Wants=network.target

[Service]
Type=forking
ExecStart=/opt/sonarqube/bin/linux-x86-64/sonar.sh start
ExecStop=/opt/sonarqube/bin/linux-x86-64/sonar.sh stop
ExecReload=/opt/sonarqube/bin/linux-x86-64/sonar.sh restart
User=sonarqube
Group=sonarqube
Restart=always
RestartSec=10
LimitNOFILE=131072
LimitNPROC=8192
TimeoutStartSec=120

[Install]
WantedBy=multi-user.target
```

Step 8: Start SonarQube

```
sudo systemctl daemon-reload
sudo systemctl enable sonarqube
sudo systemctl start sonarqube
sudo systemctl status sonarqube
```

Step 9: Monitor SonarQube Startup

```
tail -f /opt/sonarqube/logs/sonar.log
tail -f /opt/sonarqube/logs/web.log
tail -f /opt/sonarqube/logs/es.log
```

Verify port 9000:

```
sudo ss -tlnp | grep 9000
```

Step 10: Access SonarQube Web UI

- URL: `http://<sonarqube-public-ip>:9000`
- Username: `admin`
- Password: `admin`

On first login, change the default password.

Step 11: Configure SonarQube for the E-Commerce Project

Step A: Create the Project

1. Login to SonarQube
2. Click **Projects** → **Create Project** → **Manually**
3. Fill in:
   - Project display name: `ecommerce-app`
   - Project key: `ecommerce-app`
   - Main branch name: `main`
4. Click **Set Up**
5. Choose **With GitHub Actions**

Step B: Generate Authentication Token

1. Click Avatar (top-right) → **My Account**
2. Click **Security** tab
3. Under **Generate Tokens**:
   - Token Name: `github-actions-token`
   - Type: Project Analysis Token
   - Project: `ecommerce-app`
   - Expiry: `90 days`

4. Click **Generate**

Critical: Copy this token immediately. It will not be shown again. Save as `SONAR_TOKEN` in GitHub Secrets.

Step C: Configure Quality Gate

1. Go to **Quality Gates**
2. Click **Create**
3. Name: `ecommerce-quality-gate`
4. Add Conditions:
   - Coverage on New Code is less than `80%`
   - Duplicated Lines on New Code is greater than `3%`
   - Reliability Rating on New Code is worse than `A`
   - Security Rating on New Code is worse than `A`
   - Maintainability Rating on New Code is worse than `A`
5. Assign:
   - Projects → `ecommerce-app` → Project Settings → Quality Gate → select `ecommerce-quality-gate`

Step 12: Verify SonarQube is Working

```
sudo systemctl status sonarqube
curl http://localhost:9000/api/system/status
```

Expected Output (example):

```
{"id":"xxx","version":"10.6.0","status":"UP"}
```

1.7 Troubleshooting Guide

Issue 1: Node Stuck in NotReady State

```
sudo journalctl -u kubelet -f --no-pager | tail -50

# FIX A: Swap is still enabled
sudo swapoff -a
sudo systemctl restart kubelet

# FIX B: ContainerD issues
sudo systemctl restart containerd
sudo systemctl restart kubelet

# FIX C: CNI plugin issues on master
kubectl delete pod -n kube-system -l k8s-app=calico-node
```

Issue 2: Worker Cannot Join — Token Expired

```
# On master node
kubeadm token create --print-join-command

# On the worker node that failed to join
sudo kubeadm reset -f
sudo rm -rf /etc/cni/net.d
sudo rm -rf $HOME/.kube

sudo kubeadm join 172.31.1.10:6443 --token <new-token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

Issue 3: Pods Stuck in ContainerCreating (NFS Mount Failure)

```
kubectl describe pod <pod-name> -n ecommerce

# Test NFS from the worker node
showmount -e 172.31.1.13
sudo mount -t nfs 172.31.1.13:/srv/nfs/k8s-pvs/ecommerce-products /mnt
ls /mnt
sudo umount /mnt

# If mount fails, check NFS server
sudo systemctl status nfs-kernel-server
sudo exportfs -v
sudo ss -tlnp | grep 2049
```

Issue 4: SonarQube Fails to Start

```
tail -50 /opt/sonarqube/logs/sonar.log

sudo sysctl vm.max_map_count
sudo sysctl -w vm.max_map_count=524288

ps aux | grep sonar
sudo ss -tlnp | grep 9000

java -version
```

Issue 5: HAProxy Shows Backends as DOWN

```
curl http://172.31.1.11:30080/health
curl http://172.31.1.12:30080/health

sudo tail -30 /var/log/haproxy.log
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo systemctl restart haproxy
```

Issue 6: SonarQube Quality Gate Failing in Pipeline

- Review results in SonarQube UI: `http://<sonarqube-ip>:9000` → Projects → `ecommerce-app` → Issues
- Common causes: low coverage, duplications, security hotspots
- Temporary lowering of thresholds is not recommended for production.

1.8 Useful Daily Commands Reference

```
# ── CLUSTER STATUS ──────────────────────────────
kubectl get nodes -o wide
kubectl get pods -n ecommerce -o wide
kubectl get all -n ecommerce
kubectl top nodes
kubectl top pods -n ecommerce

# ── LOGS ────────────────────────────────────────
kubectl logs <pod-name> -n ecommerce
kubectl logs <pod-name> -n ecommerce --previous
kubectl logs -f <pod-name> -n ecommerce

# ── DEBUGGING ───────────────────────────────────
kubectl describe pod <pod-name> -n ecommerce
kubectl describe node worker-node-1
kubectl get events -n ecommerce --sort-by='.lastTimestamp'

# ── EXEC INTO POD ───────────────────────────────
kubectl exec -it <pod-name> -n ecommerce -- /bin/sh

# ── DEPLOYMENT MANAGEMENT ───────────────────────
kubectl rollout status deployment/ecommerce-api -n ecommerce
kubectl rollout history deployment/ecommerce-api -n ecommerce
kubectl rollout undo deployment/ecommerce-api -n ecommerce
kubectl scale deployment ecommerce-api --replicas=3 -n ecommerce

# ── NFS VERIFICATION ────────────────────────────
showmount -e 172.31.1.13
kubectl get pv
kubectl get pvc -n ecommerce

# ── SONARQUBE ───────────────────────────────────
sudo systemctl status sonarqube
tail -f /opt/sonarqube/logs/sonar.log
curl http://localhost:9000/api/system/status

# ── HAPROXY ─────────────────────────────────────
sudo systemctl status haproxy
```

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo tail -f /var/log/haproxy.log
```

1.9 Infrastructure Summary

Component	Status Check
Kubernetes Cluster	`kubectl get nodes` (all Ready) `kubectl get pods -n kube-system` (Running)
NFS Server	`showmount -e <nfs-ip>` (lists exports) `kubectl get pvc -n ecommerce` (Bound)
HAProxy	`http://<ip>:8404/stats` (dashboard loads) `curl http://<ip>/` (app responds)
SonarQube	`http://<ip>:9000` (UI loads) `systemctl status sonarqube` (active)

SECTION 2: Source Code Management

2.1 Overview

The E-Commerce application uses GitHub as the centralized version control platform for managing source code, team collaboration, pull requests, code reviews, and release history.

All microservices source code, Kubernetes manifests, and deployment configurations are maintained in Git repositories to ensure:

- Proper version tracking
- Controlled change management
- Rollback capability
- Secure collaboration
- Streamlined development workflows

2.2 GitHub Repository Details

2.2.1 Main Application Repository

Repository URL:

https://github.com/e-commerce-application-microservice/e-commerce-app.git

Repository contains:

- Frontend source code
- Backend microservices
- API Gateway
- Dockerfiles
- CI pipeline files
- Application configuration files

2.2.2 GitOps / Deployment Repository

Repository URL:

https://github.com/e-commerce-application-microservice/ecommerce-argo-cd

Repository contains:

- Helm charts
- Kubernetes manifests
- Argo CD application definitions
- Environment values files
- Rollout strategies
- Deployment automation files

2.3 Branch Strategy

A structured branching model is implemented to support parallel development, infrastructure changes, security enhancements, and stable production releases.

2.3.1 `main`

Primary stable branch used for production-ready code.

Used for:

- Final tested releases
- Approved code changes only
- Production deployments
- Stable application versions

2.3.2 `RBAC`

Dedicated branch for Kubernetes security and access control configurations.

Used for:

- `role.yaml`
- `rolebinding.yaml`
- service accounts
- namespace permissions

Purpose: Implements Role-Based Access Control (RBAC) for secure Kubernetes operations.

2.3.3 `feature/argo-blue-green`

Feature branch used for progressive deployment strategy implementation.

Used for:

- Argo Rollouts configuration

- Blue-Green deployment manifests
- Traffic switching setup
- rollback strategy

Purpose: Enables zero-downtime deployments with controlled traffic switching.

2.3.4 `feature/ci-pipeline`

Feature branch used for Continuous Integration automation.

Used for:

- GitHub Actions workflows
- Docker image build pipeline
- Security scans
- image publishing

Purpose: Automates build, test, and image publishing workflows.

2.4 Branch Workflow

Feature branches → Pull request → Code review → `main` branch → Production deployment

2.5 Microservices Overview

The application follows a microservices architecture where each service is independently developed and deployed to handle a specific business capability.

2.6 Microservices and Port Configuration

S.No	Service Name	Type	Cluster IP	External IP	Port(s)
1	admin-service	ClusterIP	None		3014/TCP
2	api-gateway	ClusterIP	10.102.149.140		3000/TCP
3	auth-service	ClusterIP	None		3002/TCP
4	cart-service	ClusterIP	None		3006/TCP
5	ecommerce-gateway	LoadBalancer	10.104.235.220		80:32367/TCP
6	inventory-service	ClusterIP	None		3005/TCP
7	invoice-service	ClusterIP	None		3010/TCP
8	mongodb	ClusterIP	None		27017/TCP
9	order-service	ClusterIP	None		3008/TCP
10	payment-service	ClusterIP	None		3009/TCP
11	product-service	ClusterIP	None		3003/TCP
12	recommendation-service	ClusterIP	None		3013/TCP
13	review-rating-service	ClusterIP	None		3012/TCP
14	search-service	ClusterIP	None		3004/TCP
15	shipping-service	ClusterIP	None		3011/TCP
16	user-service	ClusterIP	None		3001/TCP

S.No	Service Name	Type	Cluster IP	External IP	Port(s)
17	wishlist-service	ClusterIP	None		3007/TCP

2.7 External Access Configuration

Service Name: `frontend-ui`

NodePort: `30080`

2.8 Application Architecture

2.9 Infrastructure Architecture

Developers → GitHub repositories → CI pipeline (GitHub Actions) → Container registry (GHCR) → Argo CD → Kubernetes cluster → Microservices pods → Monitoring / logging / security tools

2.10 Core Infrastructure Components

Layer	Tool / Platform	Purpose
Source Control	GitHub	Code management
CI Pipeline	GitHub Actions	Build & test automation
Container Registry	GitHub Container Registry	Store Docker images
CD Tool	Argo CD	Kubernetes deployment
Containerization	Docker	Package applications
Orchestration	Kubernetes	Run containers
Monitoring	Prometheus	Metrics collection
Dashboard	Grafana	Visualization
Logging	Grafana Loki	Centralized logs
Security	Keycloak	Authentication
Ingress / LB	HAProxy	Traffic routing

2.11 Kubernetes Cluster Infrastructure

The application is deployed in a Kubernetes cluster consisting of a Control Plane node and two Worker nodes.

Control Plane node responsibilities:

- API Server
- Scheduler
- Controller Manager
- etcd

Worker node responsibilities:

- Running application pods
- Managing services
- Rollouts
- Scaling workloads

2.12 Application Infrastructure

Frontend UI → API Gateway → 15+ backend microservices → MongoDB database

2.13 Deployment Infrastructure

Continuous Integration (CI) flow:

Code push → GitHub Actions → Build Docker image → Security scan → Push image to GHCR

Continuous Delivery (CD) flow:

Image updated → Git repository manifest update → Argo CD detects change → Sync to Kubernetes → Deployment completed

---

SECTION 3: CI Pipeline

3.1 Objective

This section defines how the CI pipeline performs upstream code-quality and security gates and enforces pass/fail logic before container image build and release steps.

The CI pipeline is implemented using **GitHub Actions** and integrates:

- **SonarQube** for Static Application Security Testing (SAST) and quality gate enforcement
- **Snyk** for Software Composition Analysis (SCA) on dependencies

3.2 SAST Setup Using SonarQube

3.2.1 Prerequisites (Server)

- Linux-based system (Ubuntu recommended)
- Root or sudo access
- Minimum:
  - 2 GB RAM
  - Java 17+ (Java 21 is used here)

3.2.2 Java Installation (Server)

SonarQube requires Java to run.

```
sudo apt update
sudo apt install -y openjdk-21-jdk
```

Verify Java path:

```
readlink -f $(which java)
```

Example output:

```
/usr/lib/jvm/java-21-openjdk-amd64/bin/java
```

3.2.3 Set `JAVA_HOME` (Server)

Create environment configuration:

```
sudo nano /etc/profile.d/java.sh
```

Add:

```
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
```

Apply changes:

```
source /etc/profile.d/java.sh
```

3.2.4 Install SonarQube (Server)

Navigate to installation directory:

```
cd /opt
```

Download latest version:

```
sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-26.3.0.1
20487.zip
```

Extract and rename:

```
sudo unzip sonarqube-26.3.0.120487.zip
sudo mv sonarqube-26.3.0.120487 sonarqube
```

3.2.5 Create SonarQube User (Server)

For security, SonarQube must not run as root.

```
sudo useradd -r -m sonar
sudo chown -R sonar:sonar /opt/sonarqube
```

3.2.6 System Configuration (Server)

Edit kernel parameters:

```
sudo nano /etc/sysctl.conf
```

Add:

```
vm.max_map_count=262144
fs.file-max=65536
```

Apply configuration:

```
sudo sysctl -p
```

3.2.7 Configure SonarQube as a Service (Server)

Create systemd service file:

```
sudo nano /etc/systemd/system/sonarqube.service
```

Add the following configuration:

```
[Unit]
Description=SonarQube service
After=network.target

[Service]
Type=forking
User=sonar
Group=sonar
ExecStart=/opt/sonarqube/bin/linux-x86-64/sonar.sh start
ExecStop=/opt/sonarqube/bin/linux-x86-64/sonar.sh stop
LimitNOFILE=65536
LimitNPROC=4096
Restart=always

[Install]
WantedBy=multi-user.target
```

3.2.8 Start and Verify SonarQube (Server)

Reload systemd:

```
sudo systemctl daemon-reload
```

Enable service on boot:

```
sudo systemctl enable sonarqube
```

Start SonarQube:

```
sudo systemctl start sonarqube
```

Verify service status:

```
sudo systemctl status sonarqube
```

Expected:

- Status: active (running)

3.2.9 Access SonarQube UI (Server)

Open browser:
`http://<server-ip>:9000`

Default credentials:

- Username: `admin`
- Password: `admin`

3.3 CI Pipeline – SonarQube (SAST) Integration, Workflow and Quality Gate Enforcement

3.3.1 Branch Strategy (CI context)

- `feature/ci-pipeline` → CI testing
- `main` → production-ready code

SonarQube runs on both branches to validate code early and before production.

3.3.2 Pipeline Trigger Configuration

Workflow: `.github/workflows/ci.yml`

```
on:
  push:
    branches:
      - main
      - feature/ci-pipeline
  pull_request:
    branches:
      - main
  workflow_dispatch:
```

How the pipeline is triggered (execution flow):

1. Developer commits code
2. Code is pushed to `feature/ci-pipeline` or `main`
3. GitHub detects the event
4. GitHub Actions starts the pipeline

3.3.3 SonarQube Integration (Pipeline Job)

```
sonarqube:
  name: SonarQube Scan ($ matrix.service )
  needs: prepare
  runs-on: ubuntu-latest
  steps:
```

```yaml
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - name: SonarQube Scan
        uses: SonarSource/sonarqube-scan-action@v2
        with:
          args: >
            -Dsonar.projectKey=$ matrix.service
            -Dsonar.projectName=$ matrix.service
            -Dsonar.sources=ecommerce-microservices/$ matrix.service
        env:
          SONAR_TOKEN: $ secrets.SONAR_TOKEN
          SONAR_HOST_URL: $ secrets.SONAR_HOST_URL
```

3.3.4 SonarQube Setup (Server Side) – Project + Token + Quality Gate

Step 1: Access SonarQube

`http://<server-ip>:9000`

Step 2: Login

`admin / admin`

Step 3: Create Project

- Projects → Create Project
- Provide:
  - Project Key
  - Project Name

Step 4: Generate Token

- My Account → Security → Generate Token

Step 5: Configure Quality Gate (example rules)

- No critical vulnerabilities
- Code coverage threshold
- Maintainability rating

3.3.5 GitHub Secrets Configuration (SonarQube)

GitHub → Settings → Secrets

- `SONAR_TOKEN`
- `SONAR_HOST_URL`

3.3.6 Quality Gate Evaluation & Enforcing Pipeline Failure

Add Quality Gate Step:

```yaml
- name: SonarQube Quality Gate Check
  uses: SonarSource/sonarqube-quality-gate-action@v1
  env:
    SONAR_TOKEN: $ secrets.SONAR_TOKEN
```

Pipeline failure flow:

1. SonarQube scan completes
2. Quality Gate check runs
3. Decision:
    - PASS → pipeline continues
    - FAIL → pipeline stops

Dependency control:
`needs: sonarqube`

Ensures:

- If SonarQube fails, next jobs will not execute.
- Pipeline stops early.

3.3.7 Failure Notification (SonarQube)

```yaml
- name: Send email on failure
  if: failure()
  uses: dawidd6/action-send-mail@v3
```

If SonarQube fails:

- Email is triggered
- Contains:
    - Service name
    - Failure reason

3.4 SCA Setup Using Snyk

3.4.1 Objective

This section defines how Snyk is integrated into the CI pipeline for dependency scanning (SCA).

Snyk scans project dependencies and identifies:

- Known vulnerabilities (CVEs)
- License issues
- Outdated or insecure packages

It ensures:

- Secure dependency management

- Early detection of third-party risks
- Continuous monitoring of vulnerabilities

3.4.2 Prerequisites

- GitHub repository with CI pipeline
- Node.js installed
- Snyk account (free or enterprise)
- Access to GitHub Actions

Minimum:

- Node.js 18+ (for Node-based services)
- Internet access for vulnerability database

3.4.3 Create Snyk Account & Generate Token

Step 1: Login to Snyk

https://snyk.io

Step 2: Generate API Token

Account Settings → API Token → copy token

3.4.4 GitHub Secrets Configuration (Snyk)

GitHub → Settings → Secrets → Actions:

- `SNYK_TOKEN`

3.4.5 Snyk Integration (Pipeline Job)

```yaml
snyk:
  name: "Snyk Scan ($ matrix.service )"
  needs: [prepare, sonarqube]
  runs-on: ubuntu-latest
  strategy:
    fail-fast: false
    matrix: $ fromJson(needs.prepare.outputs.matrix)
  steps:
    - uses: actions/checkout@v4
    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: "20"
    - name: Install dependencies
      run: |
        cd ecommerce-microservices/$ matrix.service
        npm install --no-audit --no-fund
```

```
    - name: Run Snyk Scan
      uses: snyk/actions/node@master
      env:
        SNYK_TOKEN: $ secrets.SNYK_TOKEN
      with:
        args: --file=ecommerce-microservices/$ matrix.service /package.json --severi
ty-threshold=high
        command: test
```

3.4.6 Severity Threshold Enforcement

In the pipeline:

`--severity-threshold=high`

Behavior:

- If High or Critical vulnerabilities found → pipeline fails
- If only Medium/Low → pipeline continues with warning

3.4.7 Report Generation (Artifact Upload)

```
- name: Upload Snyk report
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: snyk-results-$ matrix.service
    path: snyk-*.json
```

3.4.8 Failure Flow & Notification (Snyk)

Execution logic:

1. Snyk scan completes
2. Vulnerability threshold evaluated
3. Decision:
    - PASS → pipeline continues
    - FAIL → pipeline stops

Dependency control:

`needs: sonarqube`

Ensures:

- Snyk runs only after SAST passes.
- If SonarQube fails, Snyk will not execute.

Failure notification:

```yaml
- name: Send email on failure
  if: failure()
  uses: dawidd6/action-send-mail@v3
```

Behavior (if Snyk fails):

- Email is triggered
- Contains:
  - Service name
  - Vulnerability severity
  - Failure reason

SECTION 4: Docker, Container Registry, Trivy Image Security & SMTP Alerts

Whenever developers push code changes or manually select a service for deployment, the CI pipeline builds Docker images, performs security image scans, pushes approved images to DockerHub, and sends alerts.

This section covers Security & Image Management for our E-commerce microservices platform.

4.1 Workflow Trigger Process

4.1.1 Supported triggers (actual)

The workflow runs on:

1) **Push** to branches:

- `main`
- `master`
- `feature/ci-pipeline`

2) **Pull Request** targeting:

- `main`
- `master`

3) **Manual run** via `workflow_dispatch`

4.1.2 Manual inputs (workflow_dispatch)

Input	Type	Allowed	Purpose
`service`	choice/string	one microservice name OR `all`	Release one service independently or all services in parallel
`image_tag`	string	optional (e.g., `v1.2.0`)	Overrides auto-generated tag (highest priority)

Operational note: DevOps can release a single microservice without affecting others because the pipeline prepares a per-service matrix and builds/scans/pushes each service as an isolated artifact.

```
on:
  push:
    branches:
      - main
      - master
      - feature/ci-pipeline
  pull_request:
    branches:
      - main
      - master
  workflow_dispatch:
    inputs:
      service:
        description: "Microservice to build (or all)"
        required: true
        default: "all"
      image_tag:
        description: "Optional custom image tag (e.g., v1.2.0)"
        required: false
![](file://%7B%22source%22%3A%22attachment%3A44ae954c-5678-4de0-95c2-b93645b9f0be%3A
Screenshot_2026-04-22_125446.png%22%2C%22permissionRecord%22%3A%7B%22table%22%3A%22b
lock%22%2C%22id%22%3A%2234a446aa-8af2-80ab-ab25-d30b7aaa431d%22%2C%22spaceId%22%3A%2
2cea446aa-8af2-81b6-aadc-00030293bca4%22%7D%7D)
![](file://%7B%22source%22%3A%22attachment%3Aa472fa96-6c11-4c67-8a2b-cd59ecb592b1%3A
db14a23b-f433-460d-b31f-b13c6dbed201.png%22%2C%22permissionRecord%22%3A%7B%22table%2
2%3A%22block%22%2C%22id%22%3A%2234a446aa-8af2-80c2-b0b2-fe0fd8ee8bb6%22%2C%22spaceI
d%22%3A%22cea446aa-8af2-81b6-aadc-00030293bca4%22%7D%7D)
```

4.2 Service Matrix Preparation

4.2.1 What the “prepare matrix” job does

- If `service = all`: all microservices build in parallel.
- If a single service is selected: only that service builds.

4.2.2 Why matrix is used (release/operations)

- Reduces release time through parallelism.
- Supports the microservice architecture.
- Enables isolated release of a single service.

```
jobs:
  prepare:
    runs-on: ubuntu-latest
    outputs:
      matrix: $ steps.set-matrix.outputs.matrix
```

```yaml
    steps:
      - id: set-matrix
        run: |
          if [ "$ github.event.inputs.service " = "all" ]; then
            echo 'matrix=["api-gateway","user-service","authentication-service","pro
duct-service","search-service","inventory-service","cart-service","wishlist-servic
e","order-service","payment-service","invoice-service","shipping-service","review-ra
ting-service","recommendation-service","admin-service"]' >> $GITHUB_OUTPUT
          else
            echo 'matrix=["$ github.event.inputs.service "]' >> $GITHUB_OUTPUT
          fi

  build_scan_push:
    needs: prepare
    strategy:
      fail-fast: false
      matrix:
        service: $ fromJson(needs.prepare.outputs.matrix)
```

4.3 Dockerfile Standardization

Each microservice has a dedicated Dockerfile located at:

`ecommerce-microservices/<service>/Dockerfile`

Examples:

- `ecommerce-microservices/product-service/Dockerfile`
- `ecommerce-microservices/cart-service/Dockerfile`
- `ecommerce-microservices/payment-service/Dockerfile`

Sample production Dockerfile (Node.js service):

```dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 3000
CMD ["npm","start"]
```

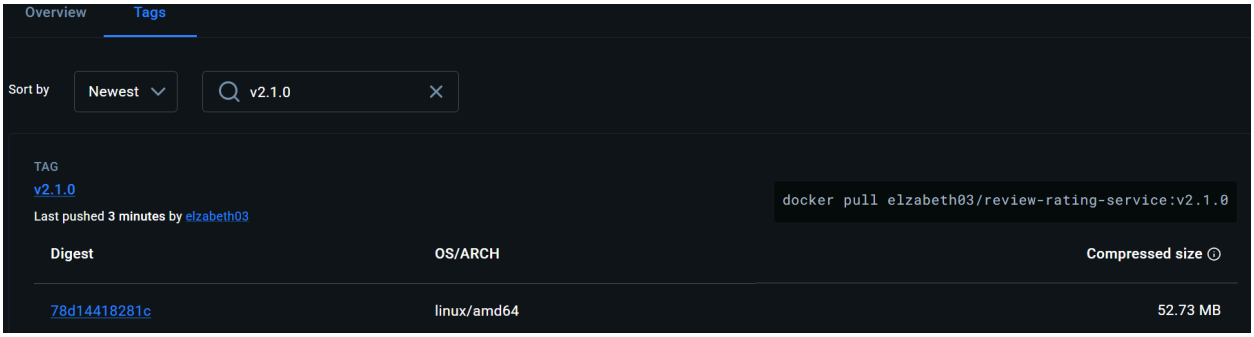
4.4 Dynamic Image Tagging Strategy

Tag resolution priority:

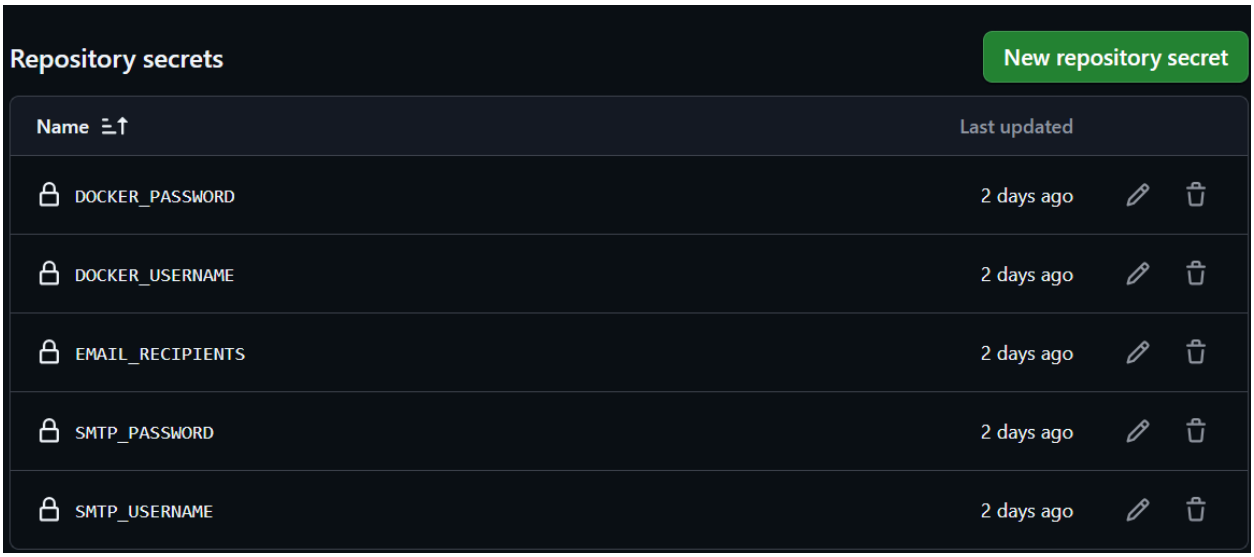
1. Manual custom tag input (`image_tag`)
2. Git release tag (e.g., `refs/tags/v2.0.1` → `v2.0.1`)
3. Git short commit hash (e.g., `3fa91bc`)

Final image examples:

- `mydockerhub/product-service:v1.2.0`
- `mydockerhub/product-service:v2.0.1`
- `mydockerhub/product-service:3fa91bc`
- `mydockerhub/product-service:latest`



4.5 Docker Build Process



Build pattern (per microservice):

```
docker build \
  -t DOCKER_USERNAME/product-service:<tag> \
  -f ecommerce-microservices/product-service/Dockerfile \
  ecommerce-microservices/product-service
```

Operational notes:

- Each selected microservice builds independently.
- `fail-fast: false` allows remaining services to continue even if one fails.

4.6 Image Artifact Handling

After build, the image is exported and uploaded as an artifact:

```
docker save IMAGE_NAME > image.tar
```

Later jobs download and load the same artifact:

```
docker load < image.tar
```

4.7 GitHub Secrets Management

Secrets used:

Secret	Purpose
`DOCKER_USERNAME`	DockerHub login + repository namespace
`DOCKER_PASSWORD`	DockerHub registry authentication
`SMTP_USERNAME`	SMTP sender authentication
`SMTP_PASSWORD`	SMTP password / app password
`EMAIL_RECIPIENTS`	recipients for alerts

4.8 Trivy Image Security Scan

Trivy runs after build and before push.

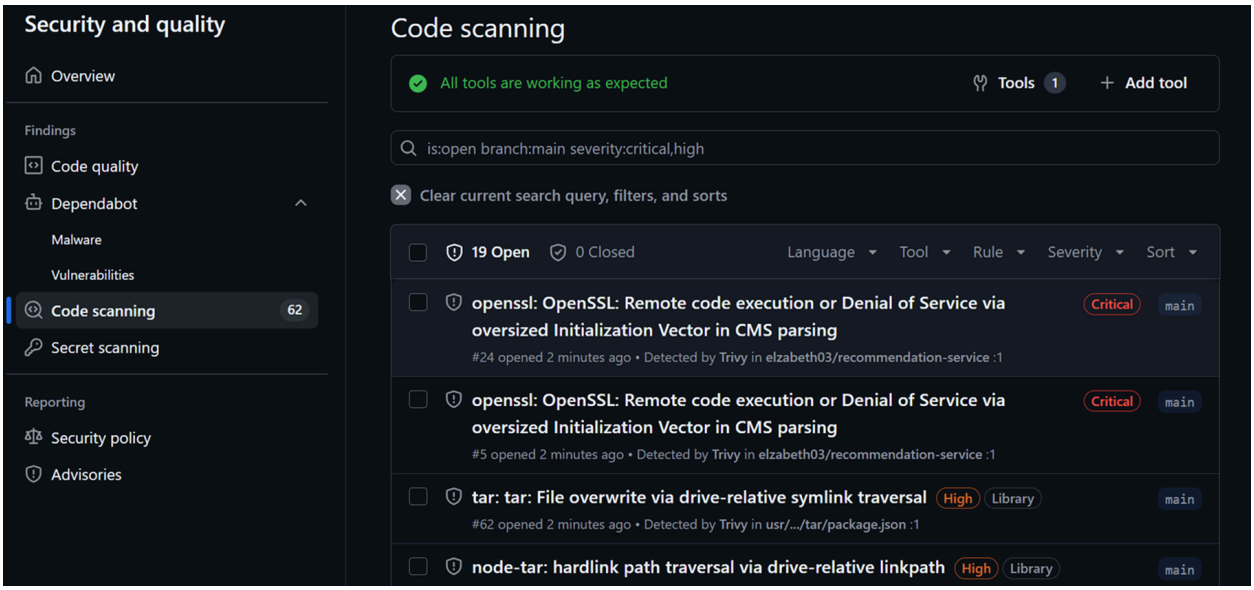
Scan settings:

- Severity: `CRITICAL,HIGH`
- Scan types: `os,library`
- Ignore unfixed: `true`
- Exit code: `1`

Sample local command:

```
trivy image mydockerhub/product-service:v1.2.0
```

If HIGH/CRITICAL vulnerabilities exist, the pipeline fails and blocks publishing.



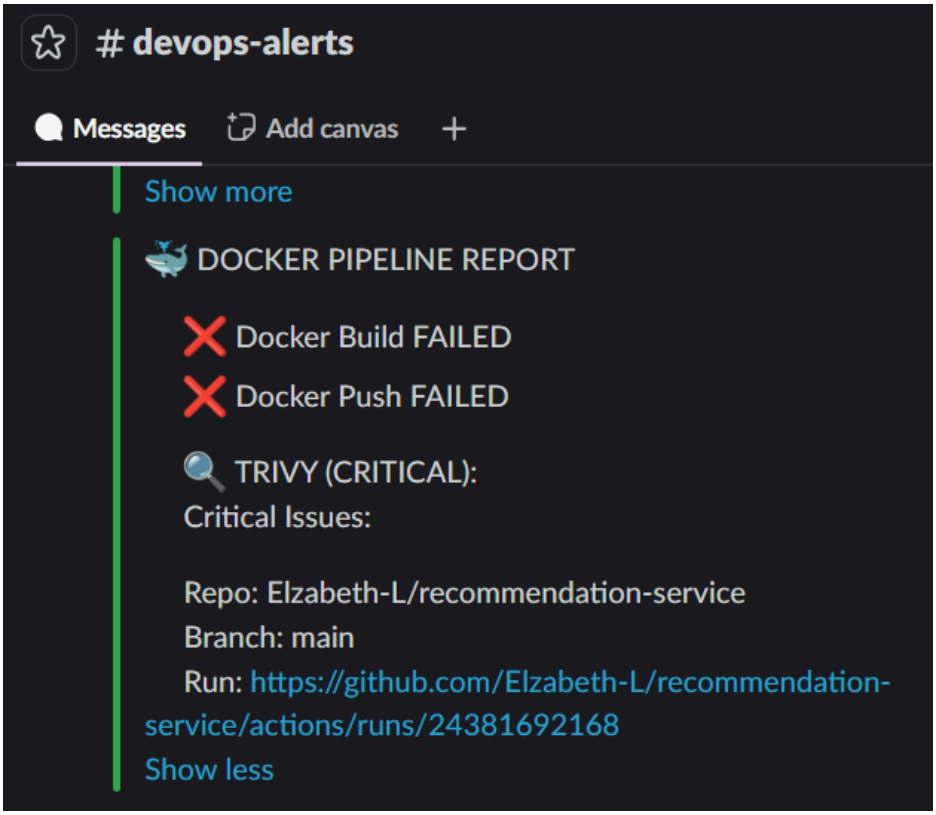
4.9 Failure Alert Notification

If Trivy fails:

- Pipeline fails for that service.
- Email alert is sent to `EMAIL_RECIPIENTS`.

Example subject:

`CI Pipeline Failed (Trivy) - product-service`



4.10 DockerHub Authentication

After successful scan:

- `docker/login-action@v3` logs into DockerHub using GitHub Secrets.

```
- name: DockerHub Login
  uses: docker/login-action@v3
  with:
    username: $ secrets.DOCKER_USERNAME
    password: $ secrets.DOCKER_PASSWORD
```

4.11 Docker Push Process

Two tags are pushed:

1) Version tag

2) `latest`

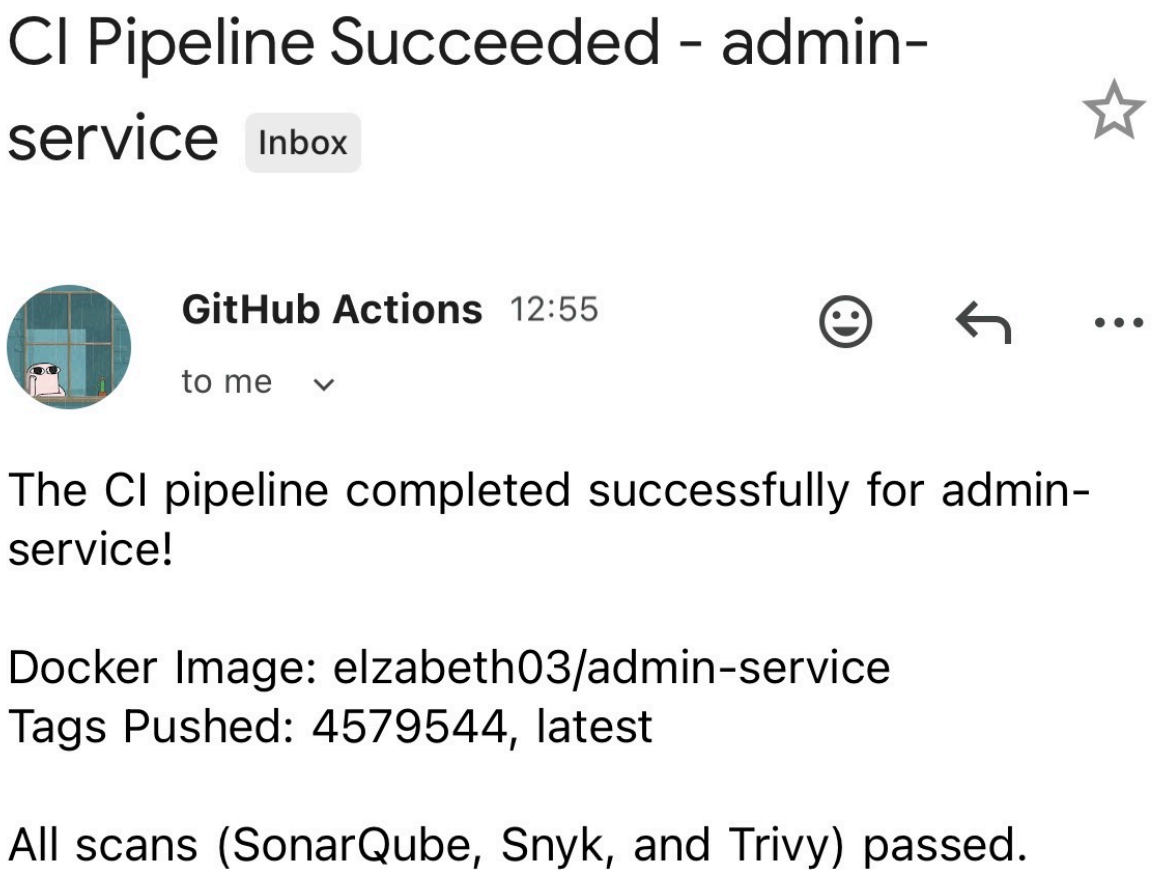
```
docker push mydockerhub/product-service:v1.2.0
docker push mydockerhub/product-service:latest
```

4.12 Final Success Notification

On successful push, a summary email is sent including:

- Service name
- Docker image
- Pushed tags
- Pipeline status

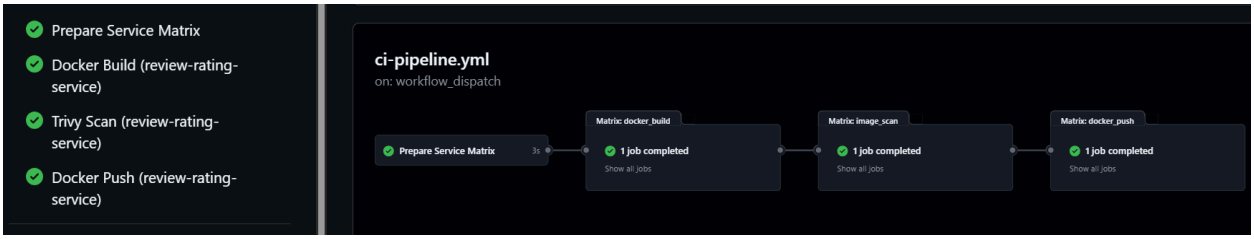
Example subject:
`CI Pipeline Succeeded - product-service`



4.13 Security Best Practices Implemented

1. Secrets stored in GitHub encrypted storage
2. No credentials in code
3. Mandatory image scan before push
4. Fail pipeline on severe vulnerabilities
5. Immutable version tagging
6. `latest` tag for active release
7. Separate image per service

8. Email alerts for failures
9. Reusable image artifacts
10. Minimal Docker base images
11. Principle of least privilege
12. Controlled registry access



4.14 Common Issues & Fixes

Issue: Docker login failed

Fix: Verify `DOCKER_USERNAME` / `DOCKER_PASSWORD`

Issue: Push denied

Fix: Confirm DockerHub repo permissions

Issue: Wrong tag generated

Fix: Verify manual `image_tag` input

Issue: Trivy failed

Fix: Patch base image OS packages or dependencies

Issue: No email received

Fix: Verify SMTP credentials and recipients

Issue: Artifact missing

Fix: Verify upload/download artifact naming consistency

4.15 End-to-End Release Flow

Developer pushes code OR manual trigger →

Prepare service matrix →

Upstream gates (SonarQube + Snyk) →

Docker Build →

Save `image.tar` →

Trivy Scan →

If fail: Email alert + stop release →

If pass: DockerHub Login →

Push image tags →

Success email →

ArgoCD consumes image for Kubernetes deployment

SECTION 5: CD Pipeline

5.0 Technical Manual: Continuous Deployment Pipeline

Scope: Argo CD + Argo Rollouts | `ecommerce-microservices/argo-deployments/`

Environment: `clahanstore` namespace | Production-Grade GitOps

5.1 Table of Contents

1. Tooling Overview
2. GitOps Storage Architecture
3. Argo CD Application Lifecycle
4. Argo Rollouts — Blue-Green Strategy Deep Dive
5. Operational Flow — End-to-End Deployment

5.2 Tooling Overview

5.2.1 Argo CD — GitOps Engine

Argo CD is the primary reconciliation engine for the entire `clahanstore` cluster state. It continuously monitors the Git repository as the declared source of truth and reconciles any divergence between the live cluster state and the committed manifests.

Capability	Purpose in This Pipeline
GitOps Sync	Detects commits to <code>argo-deployments/</code> and applies changes
Self-Heal	Reverts any out-of-band manual <code>kubectl</code> changes
Prune	Removes cluster resources no longer declared in Git
Application CRD	Defines the target repo, path, and destination namespace

5.2.2 Argo Rollouts — Progressive Delivery Controller

Argo Rollouts replaces the standard Kubernetes `Deployment` resource for the frontend service. It extends Kubernetes with CRD-based rollout strategies, enabling fine-grained traffic management that a native Deployment cannot provide.

Capability	Purpose in This Pipeline
Blue-Green Strategy	Maintains two parallel environments simultaneously
Preview Service	Routes test traffic to the new (green) version
Active Service	Routes all production traffic to the stable (blue) version
Manual Promotion	Requires explicit human approval before traffic cutover

Note: Argo CD manages the lifecycle of the Rollout resource (creates, updates, deletes it), while Argo Rollouts manages the deployment strategy within that resource. These are two distinct controllers with complementary responsibilities.

5.3 GitOps Storage Architecture

5.3.1 The Repository as the Single Source of Truth

The Git repository is not merely a code store — it is the cluster. Every resource running in the `clahanstore` namespace must have a corresponding manifest committed to the `argo-deployments/` directory. Any resource that exists in the cluster but not in Git will be pruned automatically.

```
ecommerce-microservices/
└── argo-deployments/
    ├── argocd-application.yaml      # Argo CD Application definition
    └── frontend-rollout.yaml        # Argo Rollout + Service manifests
```

5.3.2 Manifest-to-Cluster Binding

The `argocd-application.yaml` creates a direct, persistent binding between the Git repository path and the Kubernetes cluster destination. This binding is not a one-time deployment trigger — it is a continuous watch loop.

```
┌──────────────────────────────────────────────────────┐
│              argocd-application.yaml                 │
│                                                      │
│  source.repoURL  ──────────────────────────────►  | GitHub Repo
│  source.path     → argo-deployments/                 │
│  destination     → clahanstore namespace             │
└──────────────────────────────────────────────────────┘
                    │  Argo CD polls / receives webhook (~3 min)
                    ▼
┌──────────────────────────────────────────────────────┐
│           Live Kubernetes Cluster                    │
│   Namespace: clahanstore                             │
│   Resources reconciled from Git manifests            │
└──────────────────────────────────────────────────────┘
```

5.3.3 Source of Truth Enforcement Rules

Rule	Mechanism	Effect
No manual <code>kubectl apply</code>	Self-Heal	Manual changes are overwritten within seconds
No orphaned resources	Prune enabled	Resources deleted from Git are removed from cluster
No config drift	Continuous reconciliation loop	Cluster always mirrors Git state

Warning: Never run `kubectl apply -f` or `kubectl edit` directly on any resource in the `clahanstore` namespace. Argo CD's self-heal will immediately revert changes. All modifications must go through a Git commit and PR review process.

5.4 Argo CD Application Lifecycle

5.4.1 The `ecommerce-blue-green` Application Configuration

The `argocd-application.yaml` defines an Argo CD `Application` custom resource. This is the root configuration object for the entire CD pipeline.

File: `argo-deployments/argocd-application.yaml`

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: ecommerce-blue-green
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/E-Commerce-Application-Microservice/E-Commerce-App.g
it
    targetRevision: HEAD
    path: argo-deployments
  destination:
    server: https://kubernetes.default.svc
    namespace: clahanstore
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

5.4.2 Auto-Sync Behavior Explained

`automated.prune: true`

When a resource manifest is deleted from Git, Argo CD deletes the corresponding resource from the cluster on the next sync cycle.

`automated.selfHeal: true`

If a resource in `clahanstore` is modified directly (e.g., `kubectl edit`), Argo CD restores the Git-declared state within seconds.

`syncOptions: CreateNamespace=true`

The `clahanstore` namespace is auto-created if absent (idempotent on fresh clusters).

5.4.3 Application Health States (Argo CD UI)

State	Meaning
Synced	Cluster matches Git exactly
OutOfSync	Git has changes not yet applied to cluster
Progressing	Resources are being applied / rollout is in progress
Degraded	One or more resources are unhealthy
Suspended	Rollout paused awaiting manual promotion

Important: `Suspended` is expected during Blue-Green rollout: preview is live and waiting for manual verification before promotion.

5.5 Argo Rollouts — Blue-Green Strategy Deep Dive

5.5.1 Why Rollout Instead of Deployment

The `frontend-rollout.yaml` defines a `Rollout` resource rather than a standard `apps/v1 Deployment` because:

- Native Deployments perform rolling updates without traffic isolation (new pods receive production traffic immediately).
- Blue-Green Rollouts keep new pods isolated until manually promoted.
- This provides a verification window (QA/stakeholders test preview before customer traffic is switched).

5.5.2 The `frontend-rollout` Resource (Key Fields)

File: `argo-deployments/frontend-rollout.yaml`

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: frontend-rollout
  namespace: clahanstore
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: team4devops/frontend:v1.0.0
          ports:
            - containerPort: 3000
  strategy:
    blueGreen:
      activeService: frontend-active-svc
      previewService: frontend-preview-svc
      autoPromotionEnabled: false
```

5.5.3 `autoPromotionEnabled: false` — Why This Matters

This is a critical control point:

- `true`: auto-promotes after health checks (high risk: no human verification window)

- `false`: pauses at Preview until manually promoted (low risk: human control before cutover)

Important: Manual promotion is required. Verify preview on `frontend-preview-svc`, obtain sign-off, then promote. Promoting immediately defeats the Blue-Green purpose.

5.6 Operational Flow — End-to-End Deployment

5.6.1 Complete Step-by-Step Walkthrough

STEP 1: Developer commits updated image tag or manifest change to Git (PR reviewed and merged to `main`)

STEP 2: Argo CD detects `OutOfSync` and auto-sync triggers (~3 minutes or webhook)

STEP 3: Argo Rollouts creates Green ReplicaSet, binds `frontend-preview-svc` to Green, keeps production on Blue, rollout becomes `Suspended`

STEP 4: Verification window (QA tests preview, smoke/functional/visual checks, stakeholder sign-off)

STEP 5: Manual promotion executed by an authorized team member:

```
argocd app actions run ecommerce-blue-green --kind Rollout --resource-name frontend-rollout promote
```

or:

```
kubectl argo rollouts promote frontend-rollout -n clahanstore
```

STEP 6: Argo Rollouts switches `frontend-active-svc` selector to Green, rollout returns `Healthy`, old ReplicaSet retained briefly for rollback

5.6.2 Deployment State Machine (High-Level)

```
[Git Commit Merged]
        ▼
[Argo CD Detects OutOfSync]
        ▼
[Argo CD Auto-Sync Triggered]
        ▼
[Rollout Controller Creates Green ReplicaSet]
        ▼
[Preview Service → Green Pods]     [Active Service → Blue Pods (unchanged)]
        ▼
[Rollout State: SUSPENDED]
        ▼   (HUMAN VERIFICATION REQUIRED)
[Promote Command Issued]
        ▼
[Active Service → Green Pods]      [Old ReplicaSet scaled down / retained briefly]
        ▼
[Rollout State: HEALTHY]
```

SECTION 6: Monitoring, Logging & Alerts (Observability)

6.1 Overview

The observability stack is the central nervous system of our DevOps platform. It provides real-time insights into application performance, infrastructure health, and enables rapid debugging when issues arise.

We use a best-of-breed open-source stack, all deployed inside our Kubernetes cluster:

Tool	Purpose
Prometheus	Collects and stores time-series metrics
Grafana	Visualizes metrics and logs via dashboards
Loki	Aggregates and indexes logs
Promtail	Collects logs from pods and ships to Loki

6.2 Metrics with Prometheus & Grafana

6.2.1 Prometheus Setup

Prometheus is our core metrics collection engine. It dynamically discovers and scrapes metrics from all pods and nodes inside the cluster using Kubernetes service discovery.

Installation:

```
# Step 1: Add the Helm repository
helm repo add prometheus-community https://prometheus-community.github.io/helm-chart
s
helm repo update

# Step 2: Create a dedicated namespace for all monitoring tools
kubectl create namespace monitoring

# Step 3: Install the stack
helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set prometheus.ingress.enabled=true \
  --set prometheus.ingress.hosts[0]=prometheus.gathergenie.in \
  --set alertmanager.alertmanagerSpec.externalUrl=https://prometheus.gathergenie.in
```

Access:

- Internal (within cluster): `http://prometheus-operated.monitoring.svc:9090`
- External (public): `https://prometheus.gathergenie.in`

6.2.2 Grafana Setup

Grafana is our single pane of glass for all observability data. It connects to both Prometheus (for metrics) and Loki (for logs).

Installation Method:

Grafana was installed as part of the same `kube-prometheus-stack` Helm chart, so it comes pre-configured with Prometheus as a data source automatically.

```
helm install grafana grafana/grafana --namespace monitoring
```

```
kubectl patch svc grafana -n monitoring -p '{"spec": {"type": "NodePort"}}'
```

You can expose the Grafana server using the node IP and node port.

```
kubectl get secret --namespace monitoring grafana \
  -o jsonpath="{.data.admin-password}" | base64 --decode
```

Data sources configured:

Data Source	URL	Purpose
Prometheus	`http://prometheus-operated.monitoring.svc:9090`	Metrics
Loki	`http://loki.monitoring.svc:3100`	Logs

Access:

- External URL: `https://grafana.gathergenie.in`
- Login: Admin credentials stored securely in a Kubernetes Secret (auto-generated by Helm, retrieved via `kubectl get secret`)

To retrieve the Grafana admin password:

```
kubectl get secret --namespace monitoring kube-prometheus-stack-grafana \
  -o jsonpath="{.data.admin-password}" | base64 --decode
```

6.2.3 Key Dashboards

We use the following dashboards imported from the official Grafana dashboard library and customized for our stack.

Dashboard 1: Kubernetes Cluster Overview (Dashboard ID: 315 from grafana.com)

Purpose: High-level bird's-eye view of the entire cluster health.

Key panels:

Panel	Metric Used	What It Shows
Cluster CPU Usage	rate(container_cpu_usage_seconds_total[5m])	Total CPU consumed across all pods
Cluster Memory Usage	container_memory_working_set_bytes	RAM usage per namespace
Pod Status Overview	kube_pod_status_phase	Running / Pending / Failed pod counts
Node Status	kube_node_status_condition	Healthy vs NotReady nodes
Total Restarts	kube_pod_container_status_restarts_total	Crash loops indicator

Dashboard 2: Node Exporter - System Metrics (Dashboard ID: 1860 from grafana.com)

Purpose: Low-level host/node metrics like CPU, memory, disk, and network per node.

Key panels:

Panel	What It Shows
CPU Usage per Node	Per-core utilization
Memory Available	Free vs used memory
Disk I/O	Read/write throughput

Panel	What It Shows
Network Traffic	Bytes in/out per interface

Dashboard 3: Application HTTP Metrics (Custom dashboard using `kube-state-metrics` and `metrics-server` data)

Purpose: Monitor the health of our application services at the HTTP level.

Key panels:

Panel	Metric	What It Shows
Request Rate	rate(http_requests_total[5m])	Requests per second per service
Error Rate	rate(http_requests_total{status=~"5.."}[5m])	5xx errors per service
P95 Latency	histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))	95th percentile response time
Pod Restart Count	kube_pod_container_status_restarts_total	Stability indicator

6.3 Logging with Loki & Promtail

6.3.1 Promtail Setup (Log Collector)

Promtail runs as a DaemonSet — meaning exactly one Promtail pod runs on every worker node. It automatically finds and tails all container log files from the node's filesystem and ships them to Loki with enriched labels.

Installation:

```
# Add Grafana Helm repo
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update

# Install Promtail
helm install promtail grafana/promtail \
  --namespace monitoring \
  --set config.lokiAddress=http://loki.monitoring.svc:3100/loki/api/v1/push
```

6.3.2 Loki Setup (Log Aggregation)

Loki receives logs from Promtail, indexes the labels (not the full log content, which keeps it fast and cheap), and stores compressed log chunks.

Installation:

```
helm install loki grafana/loki \
  --namespace monitoring \
  --set persistence.enabled=true \
  --set persistence.size=2Gi
```

Access:

Logs are NOT accessed via Loki's API directly. All log querying is done through Grafana's **Explore** section using LogQL queries.

6.4 Headlamp Kubernetes Setup Documentation

6.4.1 Introduction

Headlamp is a web-based user interface designed to manage and monitor Kubernetes clusters. It acts as a client that communicates with the Kubernetes control plane and allows users to interact with cluster resources such as Pods, Deployments, Services, and Nodes.

Unlike traditional dashboards, Headlamp supports both:

- In-cluster access (service account-based)
- External login (token-based authentication)

Key benefits:

- Easy Kubernetes management via UI
- Visual monitoring of cluster resources
- Secure access using RBAC
- Simple deployment and access

6.4.2 Prerequisites

Before starting, ensure:

- Kubernetes cluster is running
- `kubectl` installed and configured
- Sufficient cluster permissions

6.4.3 Step 1: Create Namespace

```
kubectl create namespace headlamp
```

6.4.4 Step 2: Create Service Account

```
kubectl create serviceaccount headlamp-sa -n headlamp
```

ServiceAccount YAML (reference):

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: headlamp-sa
  namespace: headlamp
```

Explanation:

- `kind: ServiceAccount` → Defines identity for Headlamp inside Kubernetes
- `name: headlamp-sa` → Used for authentication
- `namespace: headlamp` → Scoped within Headlamp namespace

This allows Headlamp to securely communicate with the Kubernetes API.

6.4.5 Step 3: Create Cluster Role

```
kubectl create clusterrole headlamp-readonly \
  --verb=get,list,watch \
  --resource=pods,services,nodes,namespaces
```

Role definition (reference):

```
resources: ["pods", "services", "nodes", "namespaces"]
verbs: ["get", "list", "watch"]
```

Explanation:

- resources → Defines which Kubernetes objects can be accessed
- verbs → Defines allowed actions
  - get → View resource
  - list → List all resources
  - watch → Monitor changes

This ensures read-only access, improving security.

6.4.6 Step 4: Bind Role

```
kubectl create clusterrolebinding headlamp-binding \
  --clusterrole=headlamp-readonly \
  --serviceaccount=headlamp:headlamp-sa
```

6.4.7 Step 5: Deploy Headlamp

```
kubectl create deployment headlamp \
  --image=ghcr.io/headlamp-k8s/headlamp:latest \
  -n headlamp

kubectl patch deployment headlamp -n headlamp -p \
  '{"spec":{"template":{"spec":{"serviceAccountName":"headlamp-sa"}}}}'

kubectl expose deployment headlamp \
  --type=NodePort \
```

```
  --port=80 \
  --target-port=4466 \
  -n headlamp
```

6.4.8 Step 6: Verification

```
kubectl get pods -n headlamp
kubectl get svc -n headlamp
```

6.4.9 Step 7: Access UI

```
kubectl get nodes -o wide
```

Access:
`http://<NODE-IP>:<NODE-PORT>`

6.4.10 Step 8: Authentication

```
kubectl create token headlamp-sa -n headlamp
```

6.4.11 Step 9: Troubleshooting

```
kubectl logs -n headlamp deployment/headlamp
kubectl describe svc -n headlamp
kubectl auth can-i list pods --as=system:serviceaccount:headlamp:headlamp-sa
```

6.4.12 Conclusion

Headlamp simplifies Kubernetes cluster management through an intuitive UI, making it easier to monitor and manage workloads efficiently.

6.5 DAST Implementation (OWASP ZAP)

Dynamic Application Security Testing (DAST) is a security testing method that analyzes a running application to identify vulnerabilities such as SQL injection, XSS, and misconfigurations. Unlike static testing, DAST works from an external attacker's perspective.

In this setup, we use OWASP ZAP (Zed Attack Proxy), a widely used open-source tool for automated security scanning of web applications.

6.5.1 Prerequisites

Before starting, ensure:

- Ubuntu/Linux system

- Internet connectivity
- Sudo/root access
- Target application URL available

6.5.2 Scan Modes

1. Passive scan
   - Runs automatically while browsing the application
   - Does not attack the application
   - Detects issues like:
     - Missing security headers
     - Cookie misconfigurations
2. Active scan
   - Performs real attacks on the application
   - Sends payloads to detect vulnerabilities
   - Identifies:
     - SQL Injection
     - Cross-Site Scripting (XSS)
     - Command Injection

6.5.3 Install Docker

```
sudo apt update
sudo apt install docker.io
sudo service docker status
```

6.5.4 Run ZAP (Quick Scan)

```
docker run --rm -v $(pwd):/zap/wrk ghcr.io/zaproxy/zaproxy:stable zap.sh -cmd \
  quickurl https://clahanfashion.shop/ \
  -quickout /zap/wrk/zap-report.html
```

6.5.5 Check Report

```
ls
```

6.5.6 Host Report (Apache)

```
sudo apt install apache2 -y
sudo service apache2 start
sudo service apache2 status

cp ./zap-report.html /var/www/html
```

Access:

`http://<SERVER-IP>/zap-report.html`

SECTION 7: Keycloak OIDC (Kubernetes Authentication)

This section documents the production-grade OIDC authentication setup using **Keycloak** as the OIDC provider and **HAProxy** for TLS termination. It also covers Kubernetes API Server OIDC integration, client configuration, and RBAC mapping using Keycloak groups.

7.1 Architecture Overview

```
┌──────────────────────────────────────────┐
│              ARCHITECTURE FLOW           │
└──────────────────────────────────────────┘

┌──────────┐       ┌──────────────────┐       ┌──────────────────────┐
│          │       │                  │       │                      │
│  Users / │──────►│   HAProxy        │──────►│   Keycloak Server    │
│  DevOps  │ HTTPS │   (TLS Term.)    │ HTTP  │   (OIDC Provider)    │
│  Teams   │       │   Port 443       │:8080  │   auth.example.com   │
│          │       │                  │       │                      │
└──────────┘       └──────────────────┘       └──────────────────────┘
                     |                          |
                     |                          |
                  ┌──────────┐             ┌──────────┐
                  │ Certbot  │             │ PostgreSQL │
                  │ (Let's   │             │ Database   │
                  │  Encrypt) │             │            │
                  └──────────┘             └──────────┘

          ┌──────────────────────────────────────┐
          │         Kubernetes Cluster           │
kubectl   │  ┌──────────────────────────────┐   │
+ OIDC    │  │ API Server (--oidc-* flags)  │   │
 token    │  │ validates JWT tokens from Keycloak│  │
          │  └──────────────────────────────┘   │
          │  ┌──────────────────────────────┐   │
          │  │ RBAC (ClusterRoles/Bindings) │   │
          │  │ mapped to Keycloak groups    │   │
          │  └──────────────────────────────┘   │
          └──────────────────────────────────────┘
```

7.2 Component Summary

Component	Purpose	Version
Keycloak	OIDC Identity Provider	25.x+
HAProxy	Reverse Proxy + TLS Termination	2.8+
Certbot	SSL Certificate (Let's Encrypt)	Latest
PostgreSQL	Keycloak Persistent Database	15+

Component	Purpose	Version
Kubernetes	Target Cluster	1.34.5

7.3 Prerequisites

7.3.1 Infrastructure Requirements

HAProxy Server:

- OS: Ubuntu 24.04
- RAM: 4 GB minimum
- CPU: 2 vCPU
- Disk: 30 GB
- Public IP: Required (for Certbot DNS validation / HTTP challenge)

7.3.2 DNS Configuration

Create the following DNS A record before proceeding:

- `auth.example.com` → `<HAProxy_Public_IP>`

IMPORTANT: Replace `example.com` with your actual domain throughout this section. The DNS record must be propagated before SSL certificate generation.

Verify DNS propagation:

```
dig +short auth.example.com
nslookup auth.example.com
```

7.3.3 Network/Firewall Requirements

HAProxy Server:

- Inbound: `80/tcp` (Certbot HTTP challenge), `443/tcp` (HTTPS)
- Outbound: `8080/tcp` to Keycloak server

Keycloak Server:

- Inbound: `8080/tcp` from HAProxy
- Outbound: `5432/tcp` to PostgreSQL (if external DB)

Kubernetes API Server:

- Outbound: `443/tcp` to `auth.example.com` (fetch OIDC discovery)

7.3.4 Software Installation (HAProxy Server)

```
# Ubuntu/Debian
sudo apt update && sudo apt upgrade -y
sudo apt install -y haproxy certbot curl wget jq
```

```
# RHEL/CentOS
sudo dnf install -y haproxy certbot curl wget jq
```

7.3.5 Software Installation (Keycloak Server)

```
# Install Java 17 (Required for Keycloak)

# Ubuntu/Debian
sudo apt update
sudo apt install -y openjdk-17-jdk-headless wget curl unzip

# RHEL/CentOS
sudo dnf install -y java-17-openjdk-headless wget curl unzip

# Verify Java
java -version
```

7.4 SSL Certificate Generation with Certbot

7.4.1 Generate SSL Certificate (Standalone Mode)

Note: Stop HAProxy temporarily if it's already running on port 80.

```
sudo systemctl stop haproxy 2>/dev/null || true

sudo certbot certonly \
  --standalone \
  --preferred-challenges http \
  -d auth.example.com \
  --non-interactive \
  --agree-tos \
  --email devops@example.com
```

7.4.2 Verify Certificate Files

```
ls -la /etc/letsencrypt/live/auth.example.com/
```

You should see:

- `cert.pem` (server certificate)
- `chain.pem` (intermediate certificate)
- `fullchain.pem` (full chain)
- `privkey.pem` (private key)

7.4.3 Prepare Combined PEM for HAProxy

```
sudo mkdir -p /etc/haproxy/certs

sudo cat /etc/letsencrypt/live/auth.example.com/fullchain.pem \
  /etc/letsencrypt/live/auth.example.com/privkey.pem \
  | sudo tee /etc/haproxy/certs/auth.example.com.pem > /dev/null

sudo chmod 600 /etc/haproxy/certs/auth.example.com.pem
sudo chown root:root /etc/haproxy/certs/auth.example.com.pem
```

7.4.4 Setup Auto-Renewal with HAProxy Reload

Create the renewal hook script:

```
sudo tee /etc/letsencrypt/renewal-hooks/deploy/haproxy-deploy.sh << 'SCRIPT'
#!/bin/bash
DOMAIN="auth.example.com"
HAPROXY_CERT_DIR="/etc/haproxy/certs"

cat /etc/letsencrypt/live/${DOMAIN}/fullchain.pem \
  /etc/letsencrypt/live/${DOMAIN}/privkey.pem \
  > ${HAPROXY_CERT_DIR}/${DOMAIN}.pem

chmod 600 ${HAPROXY_CERT_DIR}/${DOMAIN}.pem
systemctl reload haproxy

echo "$(date): Certificate renewed and HAProxy reloaded for ${DOMAIN}" >> /var/log/c
ertbot-renewal.log
SCRIPT

sudo chmod +x /etc/letsencrypt/renewal-hooks/deploy/haproxy-deploy.sh
```

Test auto-renewal (dry run):

```
sudo certbot renew --dry-run
```

Verify timer is active:

```
sudo systemctl status certbot.timer
sudo systemctl enable certbot.timer
```

7.5 HAProxy Configuration

7.5.1 Backup Default Configuration

```
sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.backup.$(date +%F)
```

7.5.2 Configure HAProxy

```
sudo tee /etc/haproxy/haproxy.cfg << 'EOF'
#---------------------------------------------------------------------
# Global settings
#---------------------------------------------------------------------
global
  log         /dev/log local0
  log         /dev/log local1 notice
  chroot      /var/lib/haproxy
  pidfile     /var/run/haproxy.pid
  maxconn     4000
  user        haproxy
  group       haproxy
  daemon

  # SSL tuning
  ssl-default-bind-ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA25
6:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384
  ssl-default-bind-ciphersuites TLS_AES_128_GCM_SHA256:TLS_AES_256_GCM_SHA384:TLS_CH
ACHA20_POLY1305_SHA256
  ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets
  tune.ssl.default-dh-param 2048

#---------------------------------------------------------------------
# Default settings
#---------------------------------------------------------------------
defaults
  mode                    http
  log                     global
  option                  httplog
  option                  dontlognull
  option                  http-server-close
  option                  forwardfor except 127.0.0.0/8
  retries                 3
  timeout http-request    10s
  timeout queue           1m
  timeout connect         10s
  timeout client          1m
  timeout server          1m
  timeout http-keep-alive 10s
  timeout check           10s
  maxconn                 3000
```

```
#---------------------------------------------------------------------
# Frontend: HTTP → HTTPS Redirect
#---------------------------------------------------------------------
frontend http_front
  bind *:80
  mode http

  # Allow Certbot ACME challenge
  acl is_acme path_beg /.well-known/acme-challenge/
  use_backend certbot_backend if is_acme

  # Redirect all other HTTP traffic to HTTPS
  redirect scheme https code 301 if !is_acme

#---------------------------------------------------------------------
# Frontend: HTTPS (TLS Termination)
#---------------------------------------------------------------------
frontend https_front
  bind *:443 ssl crt /etc/haproxy/certs/auth.example.com.pem alpn h2,http/1.1
  mode http

  # Security headers
  http-response set-header Strict-Transport-Security "max-age=63072000; includeSubDo
mains; preload"
  http-response set-header X-Frame-Options "SAMEORIGIN"
  http-response set-header X-Content-Type-Options "nosniff"
  http-response set-header X-XSS-Protection "1; mode=block"

  # ACL: Route based on subdomain
  acl is_keycloak hdr(host) -i auth.example.com

  # Route to Keycloak backend
  use_backend keycloak_backend if is_keycloak

  # Default backend
  default_backend no_match

#---------------------------------------------------------------------
# Backend: Keycloak
#---------------------------------------------------------------------
backend keycloak_backend
  mode http
  balance roundrobin
  option httpchk GET /health/ready
  http-check expect status 200

  # Forward original client information
  http-request set-header X-Forwarded-For %[src]
```

```
  http-request set-header X-Forwarded-Proto https
  http-request set-header X-Forwarded-Host %[req.hdr(host)]
  http-request set-header X-Forwarded-Port 443

  # Keycloak server(s)
  server keycloak1 <KEYCLOAK_SERVER_IP>:8080 check inter 5s fall 3 rise 2

#---------------------------------------------------------------------
# Backend: Certbot ACME Challenge
#---------------------------------------------------------------------
backend certbot_backend
  mode http
  server certbot 127.0.0.1:8888

#---------------------------------------------------------------------
# Backend: No Match (Default deny)
#---------------------------------------------------------------------
backend no_match
  mode http
  http-request deny deny_status 403

#---------------------------------------------------------------------
# Stats Page (Optional - restrict access in production)
#---------------------------------------------------------------------
listen stats
  bind *:8404
  mode http
  stats enable
  stats uri /stats
  stats refresh 10s
  stats admin if LOCALHOST
  stats auth admin:StrongPassword123!
EOF
```

7.5.3 Validate and Start HAProxy

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo systemctl enable haproxy
sudo systemctl start haproxy
sudo systemctl status haproxy
```

7.5.4 Verify SSL Endpoint

```
curl -I https://auth.example.com
```

```
openssl s_client -connect auth.example.com:443 -servername auth.example.com </dev/nu
ll 2>/dev/null | openssl x509 -noout -dates -subject
```

7.6 Keycloak Installation

7.6.1 Install and Configure PostgreSQL

```
# Install PostgreSQL

# Ubuntu/Debian
sudo apt install -y postgresql postgresql-contrib

# RHEL/CentOS
sudo dnf install -y postgresql-server postgresql-contrib
sudo postgresql-setup --initdb

# Start and enable PostgreSQL
sudo systemctl enable postgresql
sudo systemctl start postgresql
```

Create the Keycloak database and user:

```
-- Run as: sudo -u postgres psql
CREATE USER keycloak WITH PASSWORD 'K3ycl0ak_Str0ng_P@ssw0rd!';
CREATE DATABASE keycloak OWNER keycloak;
GRANT ALL PRIVILEGES ON DATABASE keycloak TO keycloak;
```

If Keycloak is on a different server, allow the Keycloak server IP in `pg_hba.conf` and set `listen_addresses='*'` in `postgresql.conf`, then restart PostgreSQL.

7.6.2 Create Keycloak System User

```
sudo groupadd -r keycloak
sudo useradd -r -g keycloak -d /opt/keycloak -s /sbin/nologin keycloak
```

7.6.3 Download and Install Keycloak

```
KEYCLOAK_VERSION="25.0.2"
cd /tmp
wget https://github.com/keycloak/keycloak/releases/download/${KEYCLOAK_VERSION}/keyc
loak-${KEYCLOAK_VERSION}.tar.gz

sudo tar -xzf keycloak-${KEYCLOAK_VERSION}.tar.gz -C /opt/
sudo mv /opt/keycloak-${KEYCLOAK_VERSION} /opt/keycloak
sudo chown -R keycloak:keycloak /opt/keycloak
```

7.6.4 Configure Keycloak

```
sudo tee /opt/keycloak/conf/keycloak.conf << 'EOF'
# --- Database ---
db=postgres
db-url-host=localhost
db-url-port=5432
db-url-database=keycloak
db-username=keycloak
db-password=K3ycl0ak_Str0ng_P@ssw0rd!
db-schema=public

# --- HTTP ---
http-enabled=true
http-host=0.0.0.0
http-port=8080

# --- Proxy ---
proxy-headers=xforwarded
http-relative-path=/

# --- Hostname ---
hostname=auth.example.com
hostname-strict=true
hostname-strict-https=true

# --- Health ---
health-enabled=true

# --- Metrics ---
metrics-enabled=true

# --- Logging ---
log=console,file
log-level=INFO
log-file=/opt/keycloak/data/log/keycloak.log
EOF

sudo mkdir -p /opt/keycloak/data/log
sudo chown -R keycloak:keycloak /opt/keycloak/data
```

7.6.5 Build Keycloak (Optimized Production Mode)

```
sudo -u keycloak /opt/keycloak/bin/kc.sh build
```

Expected output:

- `Server configuration updated and target map file generated.`

7.6.6 Create Systemd Service

```
sudo tee /etc/systemd/system/keycloak.service << 'EOF'
[Unit]
Description=Keycloak Identity and Access Management
After=network.target postgresql.service
Requires=network.target

[Service]
Type=exec
User=keycloak
Group=keycloak
WorkingDirectory=/opt/keycloak

# Initial admin credentials (only used on first boot)
Environment="KEYCLOAK_ADMIN=admin"
Environment="KEYCLOAK_ADMIN_PASSWORD=Admin@Str0ng_P@ssw0rd!"

# Java options
Environment="JAVA_OPTS=-Xms512m -Xmx2048m -XX:MetaspaceSize=96M -XX:MaxMetaspaceSize
=256m"

ExecStart=/opt/keycloak/bin/kc.sh start --optimized
Restart=on-failure
RestartSec=10
StandardOutput=journal
StandardError=journal

# Security hardening
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/opt/keycloak/data

[Install]
WantedBy=multi-user.target
EOF
```

7.6.7 Start Keycloak

```
sudo systemctl daemon-reload
sudo systemctl enable keycloak
sudo systemctl start keycloak
```

```
sudo systemctl status keycloak
sudo journalctl -u keycloak -f --no-pager
```

7.6.8 Verify Keycloak is Running

```
curl -s http://localhost:8080/health/ready | jq .

curl -I https://auth.example.com
```

Access Admin Console:

- URL: `https://auth.example.com/admin`
- Username: `admin`
- Password: `Admin@Str0ng_P@ssw0rd!`

SECURITY: Change the admin password immediately after first login. Remove `KEYCLOAK_ADMIN` and `KEYCLOAK_ADMIN_PASSWORD` from the systemd service after initial setup.

7.7 Keycloak OIDC Configuration

7.7.1 Create a New Realm

Do **NOT** use the `master` realm for applications.

Admin Console steps:

1. Login to `https://auth.example.com/admin`
2. Dropdown (top-left) → Create Realm
3. Realm Name: `kubernetes` (Enabled: ON)

CLI (kcadm) example:

```
/opt/keycloak/bin/kcadm.sh config credentials \
  --server http://localhost:8080 \
  --realm master \
  --user admin \
  --password 'Admin@Str0ng_P@ssw0rd!'

/opt/keycloak/bin/kcadm.sh create realms \
  -s realm=kubernetes \
  -s enabled=true \
  -s displayName="Kubernetes Cluster Authentication"
```

7.7.2 Create an OIDC Client

Client:

- Client type: OpenID Connect
- Client ID: `kubernetes-cluster`

- Name: Kubernetes Cluster OIDC

Redirect URIs (for kubectl OIDC plugins):

- `http://localhost:8000/*`
- `http://localhost:18000/*`
- `urn:ietf:wg:oauth:2.0:oob`

Retrieve Client Secret:

- Clients → `kubernetes-cluster` → Credentials → copy secret

CLI example:

```
/opt/keycloak/bin/kcadm.sh create clients -r kubernetes \
  -s clientId=kubernetes-cluster \
  -s name="Kubernetes Cluster OIDC" \
  -s enabled=true \
  -s protocol=openid-connect \
  -s publicClient=false \
  -s standardFlowEnabled=true \
  -s directAccessGrantsEnabled=true \
  -s 'redirectUris=["http://localhost:8000/*","http://localhost:18000/*","urn:ietf:w
g:oauth:2.0:oob"]' \
  -s 'webOrigins=["+"]'
```

7.7.3 Create Client Scopes and Mappers

Kubernetes needs `groups` in JWT tokens.

Create client scope `groups` and add a **Group Membership** mapper:

- Token Claim Name: `groups`
- Add to ID token: ON
- Add to access token: ON
- Add to userinfo: ON

Assign `groups` scope to the client as **Default**.

(Optional) Add Username mapper:

- Token Claim Name: `preferred_username`

7.7.4 Create Groups

Create groups mapped to RBAC:

- `cluster-admins`
- `developers`
- `viewers`
- `sre-team`

7.7.5 Create Users

Create users and assign to groups (example: `john.admin` in `cluster-admins`).

7.7.6 Verify OIDC Discovery Endpoint

```
curl -s https://auth.example.com/realms/kubernetes/.well-known/openid-configuration
| jq .
```

7.7.7 Test Token Generation

```
TOKEN_RESPONSE=$(curl -s -X POST \
  "https://auth.example.com/realms/kubernetes/protocol/openid-connect/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=password" \
  -d "client_id=kubernetes-cluster" \
  -d "client_secret=${CLIENT_SECRET}" \
  -d "username=john.admin" \
  -d "password=UserStr0ngP@ss!" \
  -d "scope=openid profile email groups")

ID_TOKEN=$(echo $TOKEN_RESPONSE | jq -r .id_token)

echo $ID_TOKEN | cut -d'.' -f2 | base64 -d 2>/dev/null | jq '{preferred_username, gr
oups, iss, aud}'
```

Checkpoint: if you see `groups` in the decoded token, the OIDC configuration is correct.

7.8 Kubernetes API Server OIDC Integration

7.8.1 Ensure API Server Can Reach Keycloak

```
curl -s https://auth.example.com/realms/kubernetes/.well-known/openid-configuration
| jq .issuer

# If DNS resolution fails on control plane nodes:
echo "<HAPROXY_IP> auth.example.com" | sudo tee -a /etc/hosts
```

7.8.2 Configure API Server OIDC Flags (kubeadm clusters)

Edit:

- `/etc/kubernetes/manifests/kube-apiserver.yaml`

Add flags:

```
--oidc-issuer-url=https://auth.example.com/realms/kubernetes
--oidc-client-id=kubernetes-cluster
--oidc-username-claim=preferred_username
--oidc-username-prefix=oidc:
--oidc-groups-claim=groups
--oidc-groups-prefix=oidc:
# Optional (self-signed CA): --oidc-ca-file=/etc/kubernetes/pki/keycloak-ca.crt
```

For k3s / RKE2, add equivalent args in `/etc/rancher/k3s/config.yaml` or `/etc/rancher/rke2/config.yaml`.

7.8.3 Verify API Server Restart

```
kubectl get pods -n kube-system -l component=kube-apiserver -w
kubectl logs -n kube-system kube-apiserver-<node-name> | grep -i oidc
```

7.8.4 OIDC Flag Reference

Flag	Description	Value
<code>--oidc-issuer-url</code>	URL of the OIDC provider	<code>https://auth.example.com/realms/kubernetes</code>
<code>--oidc-client-id</code>	Client ID for the OIDC client	<code>kubernetes-cluster</code>
<code>--oidc-username-claim</code>	JWT claim to use as the username	<code>preferred_username</code>
<code>--oidc-username-prefix</code>	Prefix for OIDC usernames	<code>oidc:</code>
<code>--oidc-groups-claim</code>	JWT claim to use for groups	<code>groups</code>
<code>--oidc-groups-prefix</code>	Prefix for OIDC groups	<code>oidc:</code>

7.9 Kubeconfig & Client Configuration

7.9.1 Install kubelogin (OIDC kubectl Plugin)

```
kubectl krew install oidc-login

# Or Linux AMD64:
curl -LO https://github.com/int128/kubelogin/releases/latest/download/kubelogin_linu
x_amd64.zip
unzip kubelogin_linux_amd64.zip
sudo mv kubelogin /usr/local/bin/kubectl-oidc_login
chmod +x /usr/local/bin/kubectl-oidc_login

# macOS
brew install int128/kubelogin/kubelogin
```

```
kubectl oidc-login --help
```

7.9.2 Configure Kubeconfig for OIDC Users

```
CLUSTER_NAME="my-production-cluster"
CLUSTER_API_SERVER="https://<KUBERNETES_API_SERVER>:6443"
ISSUER_URL="https://auth.example.com/realms/kubernetes"
CLIENT_ID="kubernetes-cluster"
CLIENT_SECRET="<keycloak-client-secret>"

kubectl config set-cluster ${CLUSTER_NAME} \
  --server=${CLUSTER_API_SERVER} \
  --certificate-authority=/path/to/cluster-ca.crt \
  --embed-certs=true

kubectl config set-credentials oidc-user \
  --exec-api-version=client.authentication.k8s.io/v1beta1 \
  --exec-command=kubectl \
  --exec-arg=oidc-login \
  --exec-arg=get-token \
  --exec-arg=--oidc-issuer-url=${ISSUER_URL} \
  --exec-arg=--oidc-client-id=${CLIENT_ID} \
  --exec-arg=--oidc-client-secret=${CLIENT_SECRET} \
  --exec-arg=--oidc-extra-scope=groups \
  --exec-arg=--oidc-extra-scope=profile \
  --exec-arg=--oidc-extra-scope=email

kubectl config set-context ${CLUSTER_NAME}-oidc \
  --cluster=${CLUSTER_NAME} \
  --user=oidc-user

kubectl config use-context ${CLUSTER_NAME}-oidc
```

7.10 RBAC Policies

7.10.1 Cluster Admin RBAC (cluster-admins group)

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: oidc-cluster-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
```

```yaml
subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: Group
    name: "oidc:cluster-admins"
```

7.10.2 Developer RBAC (developers group)

(Use a custom ClusterRole and namespace-scoped RoleBindings in `development` and `staging`.)

7.10.3 Viewer RBAC (viewers group)

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: oidc-viewers
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: view
subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: Group
    name: "oidc:viewers"
```

7.10.4 Apply All RBAC Policies

```bash
kubectl apply -f cluster-admin-rbac.yaml
kubectl apply -f developer-rbac.yaml
kubectl apply -f viewer-rbac.yaml
kubectl apply -f sre-rbac.yaml

kubectl get clusterrolebindings -l app.kubernetes.io/managed-by=keycloak-oidc
kubectl get rolebindings -A -l app.kubernetes.io/managed-by=keycloak-oidc
```

7.11 Testing & Validation

7.11.1 End-to-End Login Test

```bash
kubectl auth whoami
kubectl get pods -A
```

7.11.2 Verify Token and Identity

Expected example:

- Username: `oidc:john.admin`
- Groups: `[oidc:cluster-admins system:authenticated]`

7.12 Troubleshooting

7.12.1 Debug Commands

```
# HAProxy
sudo systemctl status haproxy
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo journalctl -u haproxy -f
sudo tail -f /var/log/haproxy.log
curl http://localhost:8404/stats

# Keycloak
sudo systemctl status keycloak
sudo journalctl -u keycloak -f --since "5 minutes ago"
sudo tail -f /opt/keycloak/data/log/keycloak.log
curl -s http://localhost:8080/health/ready | jq .

# Kubernetes API Server
kubectl logs -n kube-system kube-apiserver-<node-name> | grep -i "oidc|error"

# SSL/TLS
openssl s_client -connect auth.example.com:443 -servername auth.example.com -showcer
ts </dev/null
```

7.13 Maintenance & Backup

7.13.1 Keycloak Realm Export/Backup

```
/opt/keycloak/bin/kc.sh export \
  --dir /opt/keycloak/data/export \
  --realm kubernetes \
  --users realm_file
```

7.13.2 Database Backup

```
sudo -u postgres pg_dump keycloak > /backup/keycloak_db_$(date +%F_%H%M).sql
```